



Hochschule Osnabrück

University of Applied Sciences

Fakultät

Ingenieurwissenschaften und Informatik

Bachelorarbeit

über das Thema:

**Prototypische Entwicklung eines offlinefähigen KI-basierten
Kontaktbericht-Assistenten für eine Vertriebsapp**

Autor:	Hai My Dao, Mat. -Nummer: 1057396, hai.dao@hs-osnabrueck.de
Erstprüfer:	Prof. Dr. Stephan Kleuker
Zweitprüfer:	Michael Tissen, M. Sc.
Abgabedatum:	06.04.2026

Kurzfassung

Diese Bachelorarbeit untersucht die technische Machbarkeit eines offlinefähigen, KI-basierten Kontaktbericht-Assistenten für mobile Endgeräte im Vertriebskontext. Ziel ist die prototypische Entwicklung eines Systems, das gesprochene oder frei formulierte Gesprächsinhalte automatisch analysiert und strukturierte Berichtsdaten erzeugt. Ein besonderer Fokus liegt auf der vollständigen Offline-Verarbeitung, um Datenschutzanforderungen zu erfüllen und eine Nutzung ohne Internetverbindung zu ermöglichen.

Die Umsetzung basiert auf einer modularen Architektur mit einer Flutter-basierten Benutzeroberfläche und einer nativen Swift-Komponente für Audioverarbeitung und Textanalyse. Die Integration erfolgt über das Dart Foreign Function Interface.

Die Ergebnisse zeigen, dass eine strukturierte, lokal ausgeführte Analyse von Gesprächsinhalten technisch realisierbar ist. Gleichzeitig bestehen Einschränkungen hinsichtlich Modellkomplexität und Geräteabhängigkeit. Insgesamt bestätigt die Arbeit die Machbarkeit eines datenschutzkonformen Offline-KI-Assistenten.

Abstract

This bachelor thesis investigates the technical feasibility of an offline-capable AI-based contact report assistant for mobile sales environments. The objective is the prototypical development of a system that automatically analyzes spoken or freely formulated conversation content and generates structured report data. A key focus is full offline processing to ensure data privacy and independence from internet connectivity.

The system follows a modular architecture combining a Flutter-based user interface with a native Swift component for audio processing and text analysis, integrated via Dart's Foreign Function Interface.

The results demonstrate that structured on-device analysis is technically feasible, although limitations regarding model complexity and hardware dependency remain. Overall, the thesis confirms the viability of a privacy-compliant offline AI assistant.

I Inhaltsverzeichnis

1	EINLEITUNG	1
1.1	EINFÜHRUNG IN DIE THEMATIK.....	1
1.2	ZIELE DER ARBEIT	2
1.3	AUFBAU DER ARBEIT	2
2	GRUNDLAGEN	3
2.1	SWIFT.....	3
2.1.1	<i>Typsystem und Sicherheit</i>	<i>4</i>
2.1.2	<i>Speicherverwaltung (Automatic Reference Counting).....</i>	<i>5</i>
2.1.3	<i>Interoperabilität mit C.....</i>	<i>6</i>
2.1.4	<i>Performance-Eigenschaften</i>	<i>8</i>
2.1.5	<i>Fehlerbehandlung.....</i>	<i>8</i>
2.1.6	<i>Relevanz für diese Arbeit.....</i>	<i>10</i>
2.2	FLUTTER	10
2.2.1	<i>Cross-Platform-Entwicklung</i>	<i>11</i>
2.2.2	<i>Architektur und Widget-Modell.....</i>	<i>12</i>
2.2.3	<i>Rendering-Engine.....</i>	<i>16</i>
2.2.4	<i>Performance und Entwicklungsproduktivität</i>	<i>17</i>
2.2.5	<i>Relevanz für diese Arbeit.....</i>	<i>18</i>
2.3	INTEGRATION VON FLUTTER MIT NATIVEN PLATTFORMEN	18
2.3.1	<i>Motivation und Problemstellung</i>	<i>19</i>
2.3.2	<i>Platform Channels.....</i>	<i>19</i>
2.3.3	<i>Dart Foreign Function Interface (FFI)</i>	<i>21</i>
2.3.4	<i>Vergleich der Integrationsansätze.....</i>	<i>23</i>
2.3.5	<i>Relevanz für diese Arbeit.....</i>	<i>24</i>
2.4	SPRACHVERARBEITUNG	24
2.4.1	<i>Speech-to-Text (STT)</i>	<i>24</i>
2.4.2	<i>Text-to-Speech (TTS)</i>	<i>25</i>
2.4.3	<i>Relevanz für diese Arbeit.....</i>	<i>26</i>
3	ANFORDERUNGEN AN DEN KI-BASIERTE KONTAKTBERICHT-ASSISTENTEN	27
3.1	FUNKTIONALE ANFORDERUNGEN	27
3.1.1	<i>Erfassung von Gesprächsinhalten</i>	<i>27</i>
3.1.2	<i>KI-gestützte Strukturierung von Freitext.....</i>	<i>28</i>
3.1.3	<i>Offline-Fähigkeit der Verarbeitung.....</i>	<i>28</i>
3.1.4	<i>Nachbearbeitung durch den Nutzer.....</i>	<i>29</i>
3.1.5	<i>Lokale Speicherung von Kontaktberichten</i>	<i>29</i>
3.1.6	<i>Zusammenfassung der funktionalen Anforderungen</i>	<i>29</i>
3.2	NICHT-FUNKTIONALE ANFORDERUNGEN	31
3.2.1	<i>Datenschutz und Datensicherheit</i>	<i>31</i>
3.2.2	<i>Performance</i>	<i>31</i>
3.2.3	<i>Benutzerfreundlichkeit.....</i>	<i>32</i>

3.2.4	Wartbarkeit und Erweiterbarkeit.....	32
3.2.5	Plattformintegration	32
3.2.6	Zusammenfassung der nicht-funktionalen Anforderungen	33
3.3	TECHNISCHE RAHMENBEDINGUNGEN	34
3.4	ZUSAMMENFASSUNG DER ANFORDERUNGEN	34
4	SYSTEMARCHITEKTUR.....	35
4.1	ARCHITEKTURÜBERBLICK.....	35
4.2	ARCHITEKTURSTIL UND ENTWURFSPRINZIPIEN	37
4.3	NATIVE VERARBEITUNGSLOGIK.....	38
4.3.1	AudioKit.....	39
4.3.2	KontaktberichtLogikKit	40
4.4	KOMMUNIKATION ZWISCHEN FLUTTER UND SWIFT (NATIVEN MODULEN)	41
4.5	BEWERTUNG DER ARCHITEKTUR	42
5	PROTOTYPISCHE UMSETZUNG	43
5.1	ÜBERBLICK ÜBER DIE IMPLEMENTIERUNG	43
5.2	UMSETZUNG DER BENUTZEROBERFLÄCHE IN FLUTTER	43
5.3	UMSETZUNG DER AUDIOVERARBEITUNG IN AUDIOKIT.....	46
5.4	UMSETZUNG DER KONTAKTBERICHT-LOGIK IN KONTAKTBERICHTLOGIKKIT	51
5.5	IMPLEMENTIERUNG DER FFI-INTEGRATIONSSCHICHT	54
5.6	ERSTELLUNG DER NATIVEN XCFRAMEWORK-BIBLIOTHEKEN	55
5.7	ZUSAMMENSPIEL DER SYSTEMKOMPONENTEN.....	57
5.8	ZUSAMMENFASSUNG DER PROTOTYPISCHEN UMSETZUNG.....	57
6	EVALUATION	58
6.1	EVALUATIONSMETHODIK	58
6.2	BEWERTUNG DER FUNKTIONALEN ANFORDERUNGEN	59
6.3	BEWERTUNG DER NICHT-FUNKTIONALEN ANFORDERUNGEN	61
6.4	GRENZEN DER EVALUATION	63
6.5	BEWERTUNG DER SYSTEMARCHITEKTUR.....	63
6.6	PROBLEME BEI DER IMPLEMENTIERUNG.....	66
6.7	ZUSAMMENFASSUNG DER EVALUATION.....	67
7	ZUSAMMENFASSUNG, FAZIT UND AUSBLICK.....	69
7.1	ZUSAMMENFASSUNG.....	69
7.2	FAZIT	69
7.3	AUSBLICK	70
8	LITERATURVERZEICHNIS	71
ANHANG A		74

II Abbildungsverzeichnis

ABBILDUNG 2-1: BEISPIEL FÜR OPTIONAL BINDING IN SWIFT.....	4
ABBILDUNG 2-2: BEISPIEL DER VERWENDUNG DES @_cdecl-ATTRIBUTS	7
ABBILDUNG 2-3: BEISPIEL EINER RECHENINTENSIVEN SCHLEIFENOPERATION IN SWIFT.....	8
ABBILDUNG 2-4: BEISPIEL FÜR STRUKTURIERTE FEHLERBEHANDLUNG IN SWIFT MIT THROW, TRY UND CATCH	9
ABBILDUNG 2-5: BEISPIEL EINES EINFACHEN OBERFLÄCHENAUFBAUS IN FLUTTER	13
ABBILDUNG 2-6: BEISPIEL EINES STATEFULWIDGETS MIT ZUSTANDSÄNDERUNG IN FLUTTER.....	14
ABBILDUNG 2-7: DARSTELLUNG DES VERHALTENS VON STATELESSWIDGET UND STATEFULWIDGET [14].....	15
ABBILDUNG 2-8: BEISPIEL FÜR CUSTOM RENDERING MIT FLUTTER (CANVAS-ZEICHNUNG)	16
ABBILDUNG 2-9: SCHEMATISCHER ABLAUF DER KOMMUNIKATION ZWISCHEN FLUTTER UND NATIVER PLATTFORM ÜBER PLATFORM CHANNELS	20
ABBILDUNG 2-10: METHODENAUFRUF ÜBER EINEN PLATFORM CHANNEL IN FLUTTER (DART)	20
ABBILDUNG 2-11: VERARBEITUNG EINES PLATFORM-AUFRUFS IN SWIFT	20
ABBILDUNG 2-12: SCHEMATISCHER ABLAUF EINES DIREKTEN FUNKTIONSAUFRUFS ÜBER DART FFI.....	21
ABBILDUNG 2-13: AUFRUF EINER NATIVEN FUNKTION ÜBER DART FFI	22
ABBILDUNG 2-14: C-KOMPATIBLE NATIVE FUNKTION	22
ABBILDUNG 4-1: GESAMTARCHITEKTUR DES KONTAKTBERICHT-ASSISTENTEN.....	35
ABBILDUNG 4-2: SCHNITTSTELLENMODELL ZWISCHEN FLUTTER-ANWENDUNG UND NATIVEN SWIFT-MODULEN	38
ABBILDUNG 4-3: INTERNER AUFBAU DES NATIVEN MODULS AUDIOKIT	39
ABBILDUNG 4-4: PAKETSTRUKTUR DES MODULS KONTAKTBERICHTLOGIKKIT	40
ABBILDUNG 4-5: SEQUENZDIAGRAMM DER KOMMUNIKATION ZWISCHEN FLUTTER-ANWENDUNG UND NATIVEN SWIFT-MODULEN BEI DER VERARBEITUNG EINER SPRACHEINGABE.....	41
ABBILDUNG 5-1: FREITEXTEINGABE UND START DER KI-VERARBEITUNG.....	44
ABBILDUNG 5-2: RÜCKFRAGE BEI UNKLAREN ANGABEN.....	45
ABBILDUNG 5-3: ANZEIGE EINES FERTIGEN KONTAKTBERICHTS	45
ABBILDUNG 5-4: ÜBERSICHT GESPEICHERTER KONTAKTBERICHTE	46
ABBILDUNG 5-5: KLASSENDIAGRAMM DER AUDIOVERARBEITUNG IM AUDIOKIT	47
ABBILDUNG 5-6: AUDIOSTT START()-FUNKTION	48
ABBILDUNG 5-7: AUDIOSTT STOP()-FUNKTION	49
ABBILDUNG 5-8: ZENTRALE FASSADE ZUR STEUERUNG DER AUDIOVERARBEITUNG (AUDIOKITFACADE)	49
ABBILDUNG 5-9: AKTIVITÄTSDIAGRAMM DER AUDIOVERARBEITUNG IM MODUL AUDIOKIT	50
ABBILDUNG 5-10: KLASSENDIAGRAMM DES MODULS KONTAKTBERICHTLOGIKKIT	51
ABBILDUNG 5-11: EINSTIEGSPUNKT ZUR ANALYSE VON FREITEXT-EINGABEN (KB_EXTRACT_JSON)	52
ABBILDUNG 5-12: AKTIVITÄTSDIAGRAMM DER VERARBEITUNG IM MODUL KONTAKTBERICHTLOGIKKIT.....	53
ABBILDUNG 5-13: DART-WRAPPER ZUR NUTZUNG DER NATIVEN FFI-FUNKTION ZUR FREITEXTANALYSE.....	55
ABBILDUNG 5-14: BUILD-KOMMANDOS ZUR ERSTELLUNG EINES XCFRAMEWORKS FÜR DAS MODUL AUDIOKIT	56
ABBILDUNG 6-1: QUALITATIVE BEWERTUNG DER ARCHITEKTURMERKMALE DES KONTAKTBERICHT-ASSISTENTEN	66

III Tabellenverzeichnis

TABELLE 2-1: VERGLEICH VERSCHIEDENER SPEICHERVERWALTUNGSMECHANISMEN	6
TABELLE 2-2: VERGLEICH VERSCHIEDENER ANSÄTZE DER CROSS-PLATTFORM-ENTWICKLUNG	11
TABELLE 2-3: VERGLEICH VON PLATFORM CHANNELS UND FFI	23
TABELLE 3-1: ÜBERSICHT DER FUNKTIONALEN ANFORDERUNGEN	30
TABELLE 3-2: ÜBERSICHT DER NICHT-FUNKTIONALEN ANFORDERUNGEN	33
TABELLE 4-1: ÜBERBLICK ÜBER DIE ARCHITEKTURKOMONENTEN DES SYSTEMS	36
TABELLE 6-1: BEWERTUNG DER FUNKTIONALEN ANFORDERUNGEN	60
TABELLE 6-2: BEWERTUNG DER NICHT-FUNKTIONALEN ANFORDERUNGEN	62
TABELLE 6-3: ZUORDNUNG VON ANFORDERUNGEN, ARCHITEKTURKOMONENTEN UND IMPLEMENTIERUNG	64
TABELLE 6-4: BEWERTUNG DER ARCHITEKTURMERKMALE	65

IV Abkürzungsverzeichnis

2D	Two Dimensional
ABI	Application Binary Interface
API	Application Programming Interface
ARC	Automatic Reference Counting
CSS	Cascading Style Sheets
FFI	Foreign Function Interface
GPS	Global Positioning System
HTML	HyperText Markup Language
iOS	iOS Operating System
JSON	JavaScript Object Notation
KI	Künstliche Intelligenz
LLM	Large Language Model
LLVM	Low Level Virtual Machine
NLP	Natural Language Processing
STT	Speech to Text
TTS	Text to Speech
UI	User Interface
UTF-8	Unicode Transformation Format - 8-bit
UX	User Experience

1 Einleitung

Die Digitalisierung von Geschäftsprozessen sowie der Einsatz von künstlicher Intelligenz verändern zunehmend die Arbeitsweise im Vertrieb. Ein zentraler Bestandteil der Vertriebsarbeit ist die Dokumentation von Kundengesprächen in Form von Kontaktberichten. Diese Berichte dienen dazu, Vertriebsaktivitäten nachvollziehbar festzuhalten und wichtige Gesprächsinhalte strukturiert zu erfassen.

In der Praxis erfolgt die Erstellung solcher Kontaktberichte jedoch häufig manuell. Dies ist mit einem hohen Zeitaufwand verbunden und kann zu ineffizienten Arbeitsabläufen führen.

Die folgenden Abschnitte führen in die Thematik der automatisierten Erstellung von Kontaktberichten ein. Darüber hinaus werden die Zielsetzung dieser Arbeit sowie der Aufbau der Arbeit erläutert.

1.1 Einführung in die Thematik

Die Dokumentation von Kundengesprächen ist ein wichtiger Bestandteil der Vertriebsarbeit. Nach einem Termin oder Telefonat müssen relevante Informationen häufig in Form eines Kontaktberichts festgehalten werden. In vielen Fällen erfolgt diese Dokumentation jedoch manuell, was zeitaufwendig ist und zu unvollständigen oder fehlerhaften Einträgen führen kann.

Durch Fortschritte in den Bereichen künstliche Intelligenz und Sprachverarbeitung ergeben sich neue Möglichkeiten, diesen Prozess zu unterstützen. Technologien wie Speech-to-Text ermöglichen es, gesprochene Inhalte automatisch in Text umzuwandeln und anschließend weiterzuverarbeiten. Dadurch können relevante Informationen aus Gesprächsinhalten automatisch erkannt und strukturiert dargestellt werden.

Gleichzeitig spielen Datenschutz und Offline-Fähigkeit eine wichtige Rolle, da Kontaktberichte häufig sensible Kundendaten enthalten. Vor diesem Hintergrund beschäftigt sich diese Arbeit mit der Entwicklung eines KI-basierten Assistenten, der die Erstellung von Kontaktberichten unterstützt und dabei vollständig lokal auf dem mobilen Endgerät arbeitet.

1.2 Ziele der Arbeit

Ziel dieser Bachelorarbeit ist die prototypische Entwicklung eines offlinefähigen KI-basierten Kontaktbericht-Assistenten für eine mobile Vertriebsanwendung. Der Assistent soll Vertriebsmitarbeiter bei der Erstellung strukturierter Kontaktberichte unterstützen, indem gesprochene oder frei formulierte Inhalte automatisch erfasst und analysiert werden.

Ein weiterer Schwerpunkt der Arbeit liegt auf der Umsetzung einer Architektur, die eine vollständig lokale Verarbeitung der Daten ermöglicht. Dazu wird eine Flutter-basierte Benutzeroberfläche mit nativen Swift-Modulen zur Audioverarbeitung und Textanalyse kombiniert. Die Integration zwischen beiden Komponenten erfolgt über das Dart Foreign Function Interface.

Daraus ergibt sich die zentrale Fragestellung dieser Arbeit, wie ein offlinefähiger KI-basierter Kontaktbericht-Assistent entwickelt werden kann, der eine automatisierte und strukturierte Dokumentation von Kundengesprächen ermöglicht und gleichzeitig eine datenschutzkonforme Verarbeitung gewährleistet.

Diese Arbeit präsentiert die prototypische Entwicklung eines solchen Systems und zeigt, wie eine modulare Architektur aus Flutter-Anwendung und nativen Swift-Modulen zur Sprachverarbeitung und Textanalyse umgesetzt werden kann.

1.3 Aufbau der Arbeit

Die vorliegende Arbeit ist in mehrere Kapitel gegliedert. Kapitel 2 beschreibt die grundlegenden Technologien und Konzepte, die für die Entwicklung des Systems relevant sind. Dazu gehören insbesondere die Programmiersprache Swift, das Framework Flutter sowie Mechanismen zur Integration nativer Komponenten.

Kapitel 3 definiert die funktionalen und nicht-funktionalen Anforderungen an den Kontaktbericht-Assistenten. Auf dieser Grundlage wird in Kapitel 4 die Systemarchitektur des entwickelten Prototyps vorgestellt.

Kapitel 5 beschreibt die prototypische Umsetzung der einzelnen Systemkomponenten. Abschließend werden in Kapitel 6 die Ergebnisse diskutiert und bewertet, bevor Kapitel 7 die Arbeit zusammenfasst und einen Ausblick auf mögliche zukünftige Weiterentwicklungen gibt.

2 Grundlagen

Grundlagenkapitel haben in wissenschaftlichen Arbeiten die Aufgabe, wichtige Technologien und Konzepte zu erklären. Sie zeigen auch, wie diese Themen mit der Problemstellung der Arbeit zusammenhängen. In der Informatik gehören dazu zum Beispiel Programmiersprachen, Softwarearchitekturen, Datenverarbeitung und die Interaktion zwischen Mensch und Computer.

Für die Entwicklung eines offlinefähigen KI-basierten Kontaktbericht-Assistenten sind vor allem mobile Softwareentwicklung, die Verbindung zwischen plattformübergreifenden und nativen Komponenten sowie Sprachverarbeitung wichtig.

In diesem Kapitel werden deshalb die Programmiersprache Swift, das Framework Flutter, die Verbindung zwischen Flutter und nativen Komponenten sowie Grundlagen der Sprachverarbeitung erklärt. Außerdem werden wichtige Eigenschaften wie Aufbau, Leistung und Integrationsmöglichkeiten betrachtet.

2.1 Swift

Swift ist eine von Apple entwickelte, statisch typisierte und kompilierte Programmiersprache zur Entwicklung von Anwendungen für iOS, macOS, watchOS und tvOS. Die Sprache wurde im Jahr 2014 veröffentlicht und verfolgt das Ziel, eine moderne Programmiersprache bereitzustellen, die Sicherheit, Performance und eine ausdrucksstarke Syntax kombiniert [1].

Swift ersetzt zunehmend die ältere Programmiersprache Objective-C bei der Entwicklung von Anwendungen für Apple-Plattformen. Die Sprache unterstützt verschiedene Programmieransätze, zum Beispiel objektorientierte und funktionale Konzepte. Dadurch ist eine flexible und modulare Softwareentwicklung möglich [2].

Ein wichtiges Ziel von Swift ist es, typische Fehler zu vermeiden, die in anderen Programmiersprachen auftreten können. Dazu gehören Probleme mit der Speicherverwaltung, falsche Datentypen oder fehlende Werte. Swift nutzt dafür ein strenges Typsystem, automatische Speicherverwaltung und eine klare Fehlerbehandlung [1].

Außerdem wird Swift mit dem LLVM-Compiler in nativen Maschinencode übersetzt. Dadurch kann eine hohe Ausführungsgeschwindigkeit erreicht werden. Diese Eigenschaft macht Swift

besonders geeignet für Anwendungen, die eine hohe Leistung benötigen, zum Beispiel in der Audioverarbeitung oder bei der Analyse von Signalen [3].

2.1.1 Typsystem und Sicherheit

Ein wichtiges Merkmal von Swift ist das statische Typsystem. Jeder Wert hat einen klar definierten Datentyp. Dieser kann entweder direkt angegeben werden oder wird automatisch vom Compiler bestimmt. Durch diese Typprüfung können viele Fehler schon während der Kompilierung erkannt werden, bevor das Programm ausgeführt wird [1].

Ein zentrales Sicherheitskonzept ist das sogenannte Optional-Modell. In vielen Programmiersprachen können Variablen den Wert null haben, was oft zu Laufzeitfehlern führt. Swift löst dieses Problem, indem das Fehlen eines Wertes ausdrücklich dargestellt wird [1]. Eine Variable vom Typ „String?“ kann zum Beispiel entweder einen Text oder keinen Wert enthalten [3].

Auf einen optionalen Wert kann nur zugegriffen werden, wenn er vorher geprüft und ausgepackt wird. Dadurch müssen Entwickler mögliche Fehler bewusst berücksichtigen. Das erhöht die Stabilität und Zuverlässigkeit von Anwendungen [2].

Abbildung 2-1 zeigt ein Beispiel für Optional Binding in Swift. Dabei wird geprüft, ob ein Wert vorhanden ist. Nur wenn dies der Fall ist, wird der Wert weiterverwendet. So können Laufzeitfehler vermieden werden.

```
var name: String? = "Swift"

if let value = name {
    print(value)
}
```

Abbildung 2-1: Beispiel für Optional Binding in Swift

Dieses Sprachkonzept sorgt dafür, dass mögliche Fehler früh erkannt und behandelt werden. Dadurch wird die Wahrscheinlichkeit von Laufzeitfehlern deutlich reduziert und die Robustheit der Anwendung verbessert [2].

Im Vergleich dazu erlauben Programmiersprachen wie C oder C++ den direkten Zugriff auf möglicherweise ungültige Speicherbereiche [4]. Java kennt zwar ebenfalls Nullwerte, verlangt

aber keine verpflichtende Prüfung vor der Nutzung [5]. Swift nimmt hier eine Zwischenposition ein, indem es Sicherheit zur Compile-Zeit erzwingt, ohne auf Performance zu verzichten [1].

2.1.2 Speicherverwaltung (Automatic Reference Counting)

Die Speicherverwaltung in Swift erfolgt durch Automatic Reference Counting (ARC). Dabei wird für jede Objektinstanz ein Referenzzähler geführt, der die Anzahl aktiver Referenzen auf dieses Objekt speichert. Wird eine neue Referenz erzeugt, erhöht sich der Zähler; wird eine Referenz entfernt, verringert sich dieser. Sobald der Referenzzähler den Wert null erreicht, wird der Speicher automatisch freigegeben [3] [1].

ARC unterscheidet sich konzeptionell sowohl von manueller Speicherverwaltung als auch von Garbage-Collection-Ansätzen. Während in der Programmiersprache C Speicher explizit reserviert und freigegeben werden muss, was ein hohes Fehlerpotenzial birgt, verwenden Sprachen wie Java oder C# einen Garbage Collector. Dieser durchsucht periodisch den Speicher nach nicht mehr erreichbaren Objekten, was jedoch zu nicht deterministischen Pausen führen kann [1].

Swift kombiniert deterministische Speicherfreigabe mit automatisierter Verwaltung. Dies bedeutet, dass Speicher in dem Moment freigegeben wird, in dem keine Referenzen mehr existieren. Dieser Ansatz bietet sowohl Performance-Vorteile als auch eine erhöhte Vorhersagbarkeit des Laufzeitverhaltens. Allerdings können Referenzzyklen entstehen, wenn sich zwei Objekte gegenseitig referenzieren. Swift begegnet diesem Problem durch das Konzept schwacher und unbesetzter Referenzen (weak und unowned) [1] [2].

Die verschiedenen Ansätze zur Speicherverwaltung werden in Tabelle 2.1 vergleichend gegenübergestellt.

Mechanismus	Charakteristik	Vorteil	Nachteil
Manuelle Speicherverwaltung (C)	Explizite Speicherallokation und -freigabe durch Entwickler	Maximale Kontrolle über Speicher	Hohe Fehleranfälligkeit (Memory Leaks, Dangling Pointer)
Garbage Collection (Java)	Automatische Speicherbereinigung durch Laufzeitumgebung	Geringe Entwicklerverantwortung	Nicht-deterministische Laufzeitpausen
Automatic Reference Counting (Swift)	Referenzzählung mit deterministischer Freigabe	Vorhersagbares Freigabeverhalten	Referenzzyklen möglich

Tabelle 2-1: Vergleich verschiedener Speicherverwaltungsmechanismen

Der Vergleich zeigt, dass sich die verschiedenen Ansätze zur Speicherverwaltung insbesondere hinsichtlich Automatisierungsgrad, Performance und Fehleranfälligkeit unterscheiden. Während die manuelle Speicherverwaltung in Sprachen wie C eine maximale Kontrolle über den Speicher bietet, ist sie gleichzeitig mit einem hohen Risiko für Programmierfehler verbunden. Garbage-Collection-Mechanismen automatisieren die Speicherverwaltung vollständig, können jedoch durch periodische Speicherbereinigungen zu unvorhersehbaren Laufzeitpausen führen. Automatic Reference Counting stellt einen Mittelweg zwischen diesen beiden Ansätzen dar. Durch die automatische Referenzzählung wird Speicher deterministisch freigegeben, wodurch sowohl eine hohe Performance als auch ein vorhersehbares Laufzeitverhalten erreicht werden kann [3].

2.1.3 Interoperabilität mit C

Ein weiterer zentraler Aspekt von Swift ist die Interoperabilität mit C. Swift kann C-Funktionen direkt aufrufen und selbst C-kompatible Schnittstellen bereitstellen. Diese Fähigkeit basiert auf der Unterstützung einer stabilen Application Binary Interface (ABI) sowie der Möglichkeit, Funktionen mit C-konformer Signatur zu exportieren [1] [2].

Durch diese Eigenschaft können bestehende C-Bibliotheken oder systemnahe Frameworks nahtlos eingebunden werden. Primitive Datentypen wie `Int32`, `Double` oder Zeigerstrukturen sind kompatibel mit C-Datentypen. Schnittstellen werden üblicherweise über Header-Dateien definiert, wodurch eine klare Trennung zwischen Implementierung und Zugriffsschicht entsteht [1] [3].

Abbildung 2-2 zeigt ein Beispiel für die Verwendung des `@_cdecl`-Attributs zur Bereitstellung einer C-kompatiblen Funktion in Swift.

```
@_cdecl("multiply_values")
public func multiply_values(_ a: Int32, _ b: Int32) -> Int32 {
    return a * b
}
```

Abbildung 2-2: Beispiel der Verwendung des `@_cdecl`-Attributs

Durch das Attribut `@_cdecl` wird die Swift-Funktion unter einem C-kompatiblen Namen exportiert und kann anschließend aus C oder anderen Sprachen mit FFI-Unterstützung aufgerufen werden. Die Verwendung von einfachen Datentypen wie `Int32` stellt dabei die Kompatibilität zur C-Schnittstelle sicher [1] [6].

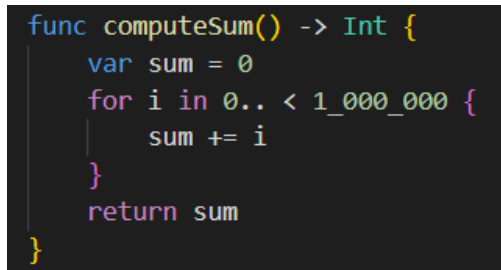
Diese Interoperabilität ist insbesondere für hybride Architekturen relevant, in denen Swift-Komponenten über ein Foreign Function Interface mit anderen Laufzeitumgebungen kommunizieren. Dadurch können native Bibliotheken aus unterschiedlichen Programmiersprachen in ein gemeinsames System integriert werden [7] [8].

Die Möglichkeit, C-kompatible Schnittstellen bereitzustellen, ist insbesondere für die Integration nativer Swift-Komponenten in plattformübergreifende Anwendungen relevant. In der vorliegenden Arbeit wird diese Eigenschaft genutzt, um Funktionen der Audio- und Sprachverarbeitung aus einer nativen Swift-Komponente über eine FFI-basierte Schnittstelle in die Flutter-Anwendung einzubinden. Dadurch kann die Benutzeroberfläche plattformübergreifend umgesetzt werden, während Performanz kritische Funktionen in nativer Umgebung ausgeführt werden [7] [8].

2.1.4 Performance-Eigenschaften

Swift wird mittels des LLVM-Compilers in nativen Maschinencode übersetzt. Dadurch erreicht die Sprache eine hohe Ausführungsgeschwindigkeit und eignet sich für rechenintensive Aufgaben wie Audioverarbeitung, Signalverarbeitung oder KI-basierte Textanalyse [1] [2]. Im Gegensatz zu interpretierten Sprachen entstehen keine zusätzlichen Laufzeitabstraktionen, wodurch Hardware-Ressourcen effizient genutzt werden können [3].

Abbildung 2-3 zeigt ein Beispiel für eine rechenintensive Schleifenoperation in Swift, bei der große Datenmengen iterativ verarbeitet werden.

A code snippet in Swift showing a function named computeSum that takes no arguments and returns an Int. The function initializes a variable sum to 0, then enters a for loop from 0 to 1,000,000, incrementing sum by i in each iteration. Finally, it returns the value of sum.

```
func computeSum() -> Int {  
    var sum = 0  
    for i in 0..  
        < 1_000_000 {  
        sum += i  
    }  
    return sum  
}
```

Abbildung 2-3: Beispiel einer rechenintensiven Schleifenoperation in Swift

Diese Funktion wird vom Compiler direkt in optimierten Maschinencode übersetzt, wodurch die Schleifenoperation effizient ausgeführt werden kann. Im Vergleich zu interpretierten Sprachen erfolgt keine zusätzliche Interpretation zur Laufzeit, was die Performance verbessert.

Die Kombination aus statischer Typisierung, deterministischer Speicherverwaltung und nativer Kompilierung positioniert Swift sowohl als leistungsfähige Anwendungssprache als auch als geeignete Grundlage für systemnahe Komponenten [1].

Diese Eigenschaften sind insbesondere für Anwendungen mit kontinuierlicher Datenverarbeitung relevant, wie beispielsweise bei der Verarbeitung von Audio- und Sprachdaten in mobilen Anwendungen.

2.1.5 Fehlerbehandlung

Swift besitzt ein strukturiertes Fehlerbehandlungssystem, das auf den Schlüsselwörtern throw, try und catch basiert. Funktionen können explizit deklarieren, dass sie Fehler auslösen können. Diese Fehler müssen anschließend explizit behandelt werden [2].

Dieses Konzept erhöht die Transparenz im Programmcode und erleichtert die Wartbarkeit komplexer Systeme.

Ein wesentliches Merkmal von Swift ist, dass Fehler explizit im Funktionsprototyp gekennzeichnet werden müssen. Dadurch wird bereits zur Compile-Zeit ersichtlich, welche Funktionen potenziell fehlschlagen können. Entwickler werden somit gezwungen, Fehler systematisch zu behandeln.

Ein Beispiel für die Fehlerbehandlung in Swift ist in Abbildung 2-4 dargestellt.

```
enum FileError: Error {  
    case notFound  
}  
  
func readFile() throws {  
    throw FileError.notFound  
}  
  
do {  
    try readFile()  
} catch {  
    print("Fehler aufgetreten")  
}
```

Abbildung 2-4: Beispiel für strukturierte Fehlerbehandlung in Swift mit throw, try und catch

In diesem Beispiel wird ein Fehler explizit über das Schlüsselwort `throw` ausgelöst und anschließend mithilfe eines `do-try-catch`-Blocks behandelt. Die Funktion `readFile()` ist als fehlerwerfend deklariert, wodurch der Aufrufer zur Fehlerbehandlung verpflichtet wird.

Im Vergleich dazu verwendet Java ebenfalls ein exception-basiertes Fehlerbehandlungssystem mit `try-catch`-Blöcken. Allerdings unterscheidet Java zwischen geprüften (`checked`) und ungeprüften (`unchecked`) Ausnahmen. Während `checked Exceptions` zwingend behandelt oder weitergereicht werden müssen, können `unchecked Exceptions` ohne explizite Behandlung auftreten, was die Vorhersagbarkeit des Programmverhaltens reduzieren kann [9].

C++ hingegen bietet ebenfalls ein exception-basiertes Modell mit `try`, `throw` und `catch`, jedoch ohne verpflichtende Kennzeichnung im Funktionskopf. Dadurch ist nicht unmittelbar ersichtlich, ob eine Funktion Fehler auslösen kann. Zudem wird die Verwendung von Exceptions in C++ in performanzkritischen Systemen häufig eingeschränkt, da deren Laufzeitverhalten weniger vorhersehbar ist [10].

Swift nimmt hier eine Zwischenposition ein: Fehler müssen explizit deklariert und behandelt werden, jedoch ohne die Komplexität eines zweistufigen Exception-Systems wie in Java. Dadurch wird eine gute Balance zwischen Sicherheit, Klarheit und Performance erreicht.

2.1.6 Relevanz für diese Arbeit

Für die Implementierung der nativen Verarbeitungslogik des Kontaktbericht-Assistenten wurde Swift gewählt, da die Sprache direkten Zugriff auf iOS-Systemressourcen ermöglicht und gleichzeitig hohe Performance mit Sicherheitsmechanismen kombiniert. Die deterministische Speicherverwaltung durch ARC unterstützt die stabile Verarbeitung von Audio- und Textdaten. Darüber hinaus erlaubt die Bereitstellung C-kompatibler Schnittstellen die Integration in eine plattformübergreifende Architektur über ein Foreign Function Interface.

Die Kombination aus Systemnähe, Performance und Sicherheit macht Swift zu einer geeigneten Grundlage für die Umsetzung nativer KI- und Audiofunktionen im Kontext dieser Arbeit.

2.2 Flutter

Flutter ist ein von Google entwickeltes Open-Source-Framework zur Entwicklung plattformübergreifender Anwendungen. Das Framework ermöglicht die Erstellung von Anwendungen für mobile Geräte, Webplattformen sowie Desktop-Systeme mit einer gemeinsamen Codebasis [7] [11].

Flutter basiert auf der Programmiersprache Dart und kombiniert ein deklaratives Benutzeroberflächenmodell mit einer eigenen Rendering-Engine [11].

Im Gegensatz zu traditionellen Cross-Platform-Ansätzen, die auf systemeigene UI-Komponenten oder Webtechnologien zurückgreifen, verwendet Flutter ein eigenständiges Rendering-System. Dadurch können Benutzeroberflächen unabhängig von der Zielplattform konsistent dargestellt werden. Anwendungen werden dabei in nativen Maschinencode kompiliert, wodurch eine hohe Ausführungsgeschwindigkeit erreicht werden kann [12] [11].

2.2.1 Cross-Platform-Entwicklung

Cross-Platform-Frameworks verfolgen das Ziel, Software für mehrere Plattformen mit einer gemeinsamen Codebasis zu entwickeln. Dadurch können Anwendungen für unterschiedliche Betriebssysteme erstellt werden, ohne dass für jede Plattform eine vollständig separate Implementierung erforderlich ist [7]. Dabei lassen sich verschiedene Ansätze unterscheiden. Native Entwicklung, hybride Ansätze und Cross-Compiled-Frameworks [7].

Bei der nativen Entwicklung wird für jede Plattform eine eigene Codebasis verwendet. Anwendungen für iOS werden beispielsweise in Swift oder Objective-C implementiert, während Android-Anwendungen typischerweise in Kotlin oder Java geschrieben werden. Dieser Ansatz ermöglicht zwar eine optimale Nutzung der jeweiligen Plattform, führt jedoch zu erhöhtem Entwicklungsaufwand, da mehrere Codebasen gepflegt werden müssen [13].

Hybride Ansätze basieren dagegen auf Webtechnologien wie HTML, CSS und JavaScript. Die Anwendung wird in einer sogenannten WebView innerhalb einer nativen Anwendung ausgeführt. Diese Architektur reduziert zwar den Entwicklungsaufwand, kann jedoch zu Einschränkungen hinsichtlich Performance und Benutzererfahrung führen [13].

Eine dritte Kategorie bilden Cross-Compiled-Frameworks. In diesem Ansatz wird eine gemeinsame Codebasis in nativen Maschinencode für die jeweilige Zielplattform kompiliert. Flutter gehört zu dieser Kategorie, da der Dart-Code während des Build-Prozesses in nativen Code übersetzt wird. Dadurch verbindet Flutter die Vorteile plattformübergreifender Entwicklung mit der Performance nativer Anwendungen [12] [11].

Entwicklungsansatz	Beschreibung	Vorteile	Nachteile
Native Entwicklung	Separate Implementierung für jede Plattform	Maximale Performance, direkter Zugriff auf System-APIs	Hoher Entwicklungsaufwand
Hybrid (WebView)	Webtechnologien in WebView	Schnelle Entwicklung	Performance- und UX-Einschränkungen
Cross-Compiled Frameworks (Flutter)	Gemeinsamer Code wird in nativen Code kompiliert	Gute Performance bei reduzierter Entwicklungszeit	Abhängigkeit vom Framework

Tabelle 2-2: Vergleich verschiedener Ansätze der Cross-Plattform-Entwicklung

Im Vergleich zu hybriden Ansätzen ermöglicht Flutter durch die Kompilierung in nativen Maschinencode eine deutlich höhere Ausführungsgeschwindigkeit. Gleichzeitig reduziert die gemeinsame Codebasis den Entwicklungsaufwand im Vergleich zu vollständig nativer Entwicklung. Dadurch stellt Flutter einen Kompromiss zwischen Entwicklungsproduktivität und Performance dar [7].

2.2.2 Architektur und Widget-Modell

Ein zentrales Konzept von Flutter ist das Widget-Modell. In Flutter wird die gesamte Benutzeroberfläche aus sogenannten Widgets aufgebaut. Ein Widget beschreibt dabei ein einzelnes Element der Benutzeroberfläche, beispielsweise Text, Layoutstrukturen oder interaktive Komponenten [11] [7].

Die Architektur folgt einem deklarativen Programmierparadigma, bei dem die Benutzeroberfläche als Funktion des aktuellen Zustands beschrieben wird. Änderungen im Anwendungszustand führen automatisch zu einer Aktualisierung der Benutzeroberfläche. Dadurch wird die Konsistenz zwischen Datenmodell und Darstellung gewährleistet [12] [11].

Widgets können grundsätzlich in zwei Kategorien unterteilt werden. Zustandslose und zustandsbehaftete Widgets. Ein `StatelessWidget` beschreibt eine unveränderliche Benutzeroberfläche, deren Darstellung ausschließlich von den übergebenen Parametern abhängt. Ein `StatefulWidget` hingegen besitzt einen internen Zustand, der sich während der Laufzeit ändern kann. Änderungen dieses Zustands führen dazu, dass die betroffene Widget-Hierarchie neu aufgebaut wird [11].

Ein Beispiel für den Aufbau einer einfachen Benutzeroberfläche mit Flutter ist in Abbildung 2-5 dargestellt.

```
class MyHomePage extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(  
        title: Text('Kontaktbericht'),  
      ),  
      body: Center(  
        child: Column(  
          mainAxisAlignment: MainAxisAlignment.center,  
          children: [  
            Text('Neuer Bericht'),  
            ElevatedButton(  
              onPressed: () {},  
              child: Text('Start Aufnahme'),  
            ),  
          ],  
        ),  
      ),  
    );  
  }  
}
```

Abbildung 2-5: Beispiel eines einfachen Oberflächenaufbaus in Flutter

Abbildung 2-5 zeigt den deklarativen Aufbau einer Benutzeroberfläche in Flutter. Die Oberfläche besteht aus einer Hierarchie von Widgets, beginnend mit dem Scaffold als strukturellem Grundgerüst. Innerhalb des body-Bereichs wird ein Column-Widget verwendet, um mehrere UI-Elemente vertikal anzuordnen. Dazu zählen ein Text-Widget zur Anzeige sowie ein interaktiver Button (ElevatedButton), der eine Benutzeraktion repräsentiert.

Dieses Beispiel verdeutlicht, wie Layout und Funktionalität durch die Verschachtelung von Widgets beschrieben werden.

Im Gegensatz dazu zeigt Abbildung 2-6 ein zustandsbehaftetes Widget, bei dem sich Inhalte dynamisch verändern können.

```
class CounterWidget extends StatefulWidget {
  @override
  _CounterWidgetState createState() => _CounterWidgetState();
}

class _CounterWidgetState extends State<CounterWidget> {
  int counter = 0;

  void increment() {
    setState(() {
      counter++;
    });
  }

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        Text('Zähler: $counter'),
        ElevatedButton(
          onPressed: increment,
          child: Text('Erhöhen'),
        ),
      ],
    );
  }
}
```

Abbildung 2-6: Beispiel eines StatefulWidgets mit Zustandsänderung in Flutter

Abbildung 2-6 zeigt ein Beispiel für ein zustandsbehaftetes Widget, bei dem sich der dargestellte Wert dynamisch verändert. Der Zustand wird in der Klasse `_CounterWidgetState` gespeichert. Durch den Aufruf von `setState()` wird eine Änderung des Zustands signalisiert, woraufhin das Widget neu aufgebaut wird. Dieses Prinzip ist zentral für das reaktive UI-Modell von Flutter, da Änderungen automatisch in der Benutzeroberfläche reflektiert werden.

Der grundlegende Unterschied zwischen zustandslosen und zustandsbehafteten Widgets wird in Abbildung 2-7 dargestellt.

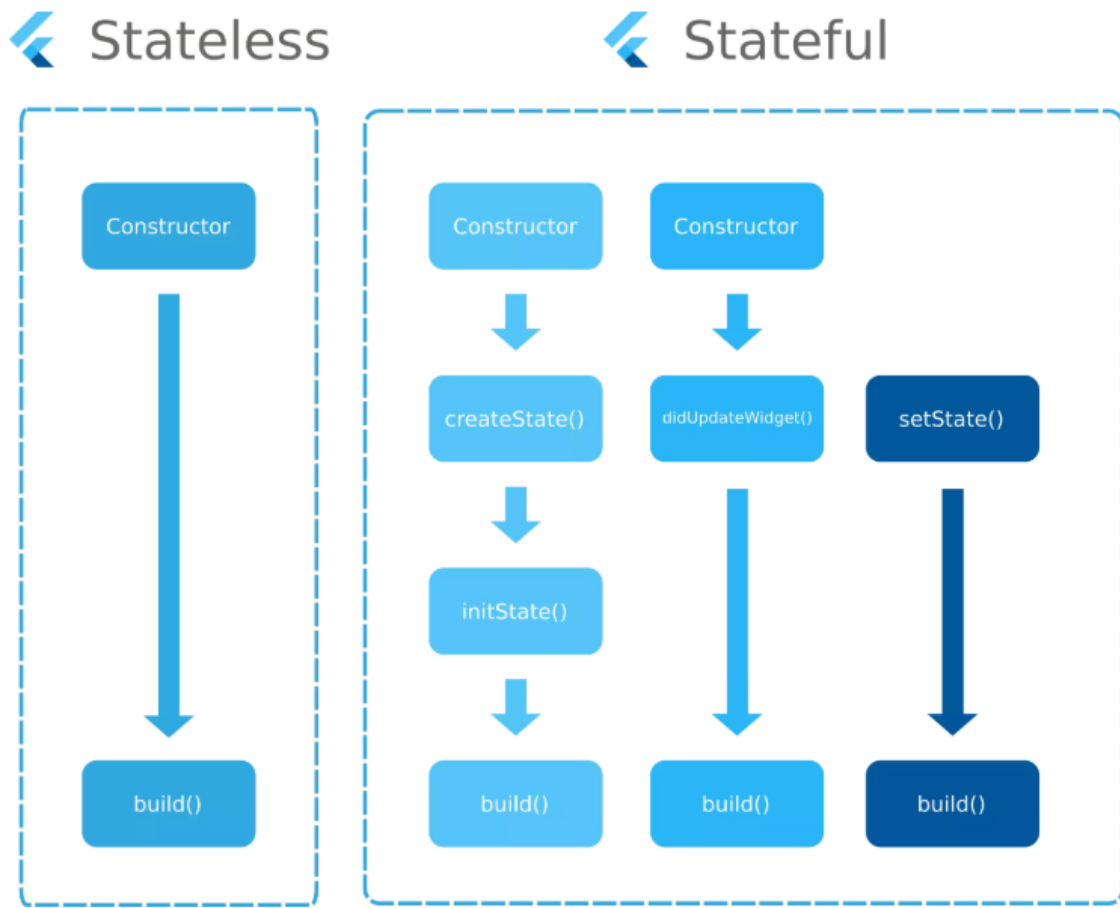


Abbildung 2-7: Darstellung des Verhaltens von StatelessWidget und StatefulWidget [14]

Wie in Abbildung 2-7 dargestellt, unterscheidet sich ein StatefulWidget von einem StatelessWidget insbesondere durch die Möglichkeit, den internen Zustand über `setState()` zu verändern, wodurch eine Aktualisierung der Benutzeroberfläche ausgelöst wird. Ein StatelessWidget hingegen besitzt keinen veränderlichen Zustand und wird ausschließlich durch die `build()`-Methode beschrieben [14].

Dieses reaktive Architekturmodell erleichtert die Wartbarkeit von Anwendungen, da Änderungen im Zustand automatisch in der Benutzeroberfläche berücksichtigt werden. Eine klare Trennung von Darstellung und Zustandslogik ist jedoch nicht grundsätzlich vorgegeben, sondern kann durch den Einsatz geeigneter Architekturmuster zusätzlich erreicht werden [7].

2.2.3 Rendering-Engine

Ein wesentliches Merkmal von Flutter ist die Verwendung einer eigenen Rendering-Engine. Während viele Cross-Platform-Frameworks systemeigene UI-Komponenten verwenden, rendert Flutter seine Benutzeroberfläche selbst. Grundlage dafür bildet die Grafikbibliothek Skia, eine von Google entwickelte 2D-Grafikbibliothek zur plattformunabhängigen Darstellung grafischer Inhalte [11] [7]. In neueren Flutter-Versionen kommt zusätzlich die Rendering-Engine Impeller zum Einsatz, die speziell für Flutter entwickelt wurde und eine verbesserte Performance sowie stabilere Darstellung ermöglicht [15].

Ein einfaches Beispiel für das eigenständige Rendering in Flutter ist die direkte Zeichnung grafischer Elemente über die Rendering-Engine, wie in Abbildung 2-8 dargestellt.

```
class MyPainter extends CustomPainter {  
  @override  
  void paint(Canvas canvas, Size size) {  
    final paint = Paint()  
      ..color = Colors.blue  
      ..strokeWidth = 4;  
  
    // Zeichnet eine Linie  
    canvas.drawLine(  
      Offset(0, 0),  
      Offset(size.width, size.height),  
      paint,  
    );  
  }  
  
  @override  
  bool shouldRepaint(CustomPainter oldDelegate) => false;  
}  
  
class DrawingExample extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(title: Text("Rendering Beispiel")),  
      body: CustomPaint(  
        size: Size(double.infinity, double.infinity),  
        painter: MyPainter(),  
      ),  
    );  
  }  
}
```

Abbildung 2-8: Beispiel für Custom Rendering mit Flutter (Canvas-Zeichnung)

Abbildung 2-8 zeigt ein Beispiel für die direkte Nutzung der Flutter-Rendering-Engine. Mit Hilfe des Canvas-Objekts werden grafische Elemente, wie beispielsweise eine Linie, explizit gezeichnet. Dieses Vorgehen verdeutlicht, dass Flutter die Darstellung der Benutzeroberfläche nicht über native UI-Komponenten realisiert, sondern eigenständig über die Skia-basierte Rendering-Engine rendert. Dadurch erhält die Anwendung eine hohe Kontrolle über grafische Inhalte sowie eine konsistente Darstellung über verschiedene Plattformen hinweg.

Durch diese Architektur kann Flutter die Darstellung der Benutzeroberfläche vollständig kontrollieren. Dadurch werden Unterschiede zwischen verschiedenen Betriebssystemen minimiert und eine konsistente Benutzererfahrung ermöglicht. Gleichzeitig erlaubt diese Architektur eine präzise Kontrolle über Layout, Animationen und grafische Effekte [12].

Im Gegensatz zu Hybridlösungen, bei denen Benutzeroberflächen innerhalb einer WebView gerendert werden, erfolgt die Darstellung in Flutter direkt über die Rendering-Engine. Dadurch werden zusätzliche Abstraktionsschichten vermieden, was sich positiv auf die Performance auswirkt [11] [13].

2.2.4 Performance und Entwicklungsproduktivität

Flutter-Anwendungen werden nicht interpretiert, sondern während des Build-Prozesses in nativen Maschinencode kompiliert. Dadurch kann eine hohe Ausführungsgeschwindigkeit erreicht werden und es ermöglicht flüssige Animationen sowie reaktionsschnelle Benutzeroberflächen [12] [11].

Ein weiterer Vorteil des Frameworks ist die sogenannte Hot Reload-Funktion. Diese ermöglicht es, Änderungen am Quellcode während der Entwicklung unmittelbar in der laufenden Anwendung sichtbar zu machen, ohne die Anwendung vollständig neu starten zu müssen. Dadurch verkürzt sich der Entwicklungszyklus erheblich und die Produktivität von Entwicklerinnen und Entwicklern wird gesteigert [7] [11].

2.2.5 Relevanz für diese Arbeit

Für die prototypische Umsetzung des Kontaktbericht-Assistenten wurde Flutter zur Implementierung der Benutzeroberfläche eingesetzt. Das Framework ermöglicht eine schnelle Entwicklung moderner Benutzeroberflächen und unterstützt gleichzeitig die Integration nativer Komponenten.

Insbesondere die Möglichkeit, Flutter mit nativen Plattformfunktionen zu kombinieren, ist für diese Arbeit von zentraler Bedeutung. Während die Benutzeroberfläche in Flutter realisiert wird, erfolgt die Verarbeitung von Audio- und KI-Funktionen in einer nativen Swift-Komponente. Durch diese Architektur können die Vorteile plattformübergreifender Entwicklung mit der Leistungsfähigkeit nativer Implementierungen kombiniert werden.

2.3 Integration von Flutter mit nativen Plattformen

Eine wichtige Eigenschaft von Flutter ist die Möglichkeit, mit nativen Plattformen zu interagieren. Dadurch können Funktionen genutzt werden, die nicht direkt durch das Framework bereitgestellt werden, beispielsweise System-APIs, Audioverarbeitung oder spezialisierte KI-Bibliotheken. In vielen Fällen greifen mobile Anwendungen auf systemnahe Funktionen zu, die ausschließlich über native Programmierschnittstellen verfügbar sind [11] [7].

Flutter-Anwendungen werden in der Programmiersprache Dart implementiert. Viele plattform-spezifische Funktionen stehen jedoch nur in nativen Sprachen wie Swift, Objective-C, Kotlin oder Java zur Verfügung. Daher ist eine Integrationsschicht erforderlich, die die Kommunikation zwischen Flutter und den nativen Komponenten ermöglicht. Diese ermöglicht die Kommunikation zwischen der Flutter-Anwendung und nativen Komponenten des jeweiligen Betriebssystems [12].

Flutter stellt hierfür mehrere Mechanismen bereit, die unterschiedliche Integrationsstrategien unterstützen. Die wichtigsten Ansätze sind sogenannte Platform Channels sowie das Foreign Function Interface (FFI). Beide Mechanismen erlauben eine Verbindung zwischen Dart-Code und nativen Bibliotheken, unterscheiden sich jedoch hinsichtlich ihres Kommunikationsmodells, ihrer Performance sowie ihrer Komplexität [11] [8] [16].

2.3.1 Motivation und Problemstellung

Obwohl Flutter eine plattformübergreifende Entwicklung ermöglicht, existieren zahlreiche Funktionen eines Betriebssystems, die nicht direkt durch das Framework bereitgestellt werden. Dazu gehören insbesondere systemnahe Schnittstellen wie Audioaufnahme, Hardwarezugriffe oder spezialisierte Plattformbibliotheken [11] [7].

Diese Funktionen sind typischerweise ausschließlich über native APIs verfügbar. Im Fall von iOS erfolgt der Zugriff beispielsweise über Programmierschnittstellen, die in Swift oder Objective-C implementiert sind. Daraus ergibt sich die Notwendigkeit einer Integrationsschicht, die eine Kommunikation zwischen dem Dart-Code der Flutter-Anwendung und den nativen Komponenten des Betriebssystems ermöglicht [12] [8].

2.3.2 Platform Channels

Platform Channels stellen einen Mechanismus zur Kommunikation zwischen Flutter und der nativen Plattform dar. Dabei werden strukturierte Nachrichten zwischen der Dart-Laufzeitumgebung und der jeweiligen nativen Plattform ausgetauscht [11] [7].

Der Ablauf einer Kommunikation über Platform Channels erfolgt typischerweise in mehreren Schritten. Zunächst sendet die Flutter-Anwendung eine Nachricht über einen definierten Kanal an die native Plattform. Diese Nachricht wird anschließend von einer entsprechenden Implementierung in der jeweiligen nativen Programmiersprache verarbeitet. Das Ergebnis kann danach als Antwort über denselben Kanal an die Flutter-Anwendung zurückgegeben werden [12].

Ein wesentliches Merkmal dieses Ansatzes ist die Serialisierung der übertragenen Daten. Informationen werden vor der Übertragung in plattformunabhängige Datenstrukturen konvertiert, wodurch eine zusätzliche Abstraktionsschicht zwischen Dart und der nativen Plattform entsteht. Dieser Mechanismus erleichtert zwar die Integration nativer APIs, führt jedoch zu zusätzlichem Kommunikationsaufwand [11].

Der grundsätzliche Kommunikationsablauf über Platform Channels ist in Abbildung 2-9 schematisch dargestellt. Dabei wird eine Nachricht aus der Flutter-Anwendung über einen definierten Kanal an die native Plattform gesendet, dort verarbeitet und anschließend als Ergebnis an Flutter zurückgegeben.

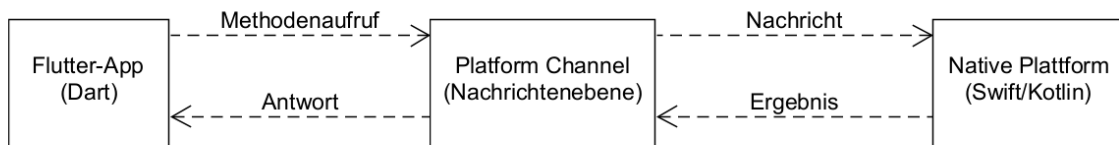


Abbildung 2-9: Schematischer Ablauf der Kommunikation zwischen Flutter und nativer Plattform über Platform Channels

Abbildung 2-9 zeigt den grundlegenden Ablauf der Kommunikation über Platform Channels. Die Flutter-Anwendung sendet dabei einen Methodenaufruf aus der Dart-Laufzeitumgebung an einen definierten Kanal. Die übermittelten Daten werden serialisiert und auf der nativen Plattform durch eine entsprechende Implementierung verarbeitet. Anschließend wird das Ergebnis über denselben Kommunikationskanal an die Flutter-Anwendung zurückgegeben. Die Abbildung verdeutlicht, dass zwischen Dart und der nativen Plattform eine zusätzliche Abstraktions- und Kommunikationsschicht besteht.

Ein vereinfachtes Beispiel für einen Methodenaufruf über einen Platform Channel ist in Abbildung 2-10 (Flutter-Seite) und Abbildung 2-11 (native Implementierung in Swift) dargestellt.

```
static const platform = MethodChannel('com.example/device');

Future<String> getBatteryLevel() async {
  final String result = await platform.invokeMethod('getBatteryLevel');
  return result;
}
```

Abbildung 2-10: Methodenaufruf über einen Platform Channel in Flutter (Dart)

```
channel.setMethodCallHandler { call, result in
  if call.method == "getBatteryLevel" {
    result("85%")
  } else {
    result(FlutterMethodNotImplemented)
  }
}
```

Abbildung 2-11: Verarbeitung eines Platform-Aufrufs in Swift

In diesem Beispiel sendet die Flutter-Anwendung den Methodenaufruf `getBatteryLevel()` an die native Plattform. Die native Implementierung verarbeitet den Methodenaufruf und gibt den Batteriestand als Ergebnis an Flutter zurück. Dadurch wird das Prinzip des nachrichtenbasierten Datenaustauschs zwischen Dart und nativer Laufzeitumgebung exemplarisch verdeutlicht.

Aus diesem Grund eignen sich Platform Channels besonders für ereignisbasierte Kommunikation oder für den Zugriff auf einzelne Systemfunktionen, beispielsweise Kamera-, GPS- oder Sensorzugriffe. Für rechenintensive Operationen kann dieser Ansatz jedoch ineffizient sein, da jeder Funktionsaufruf einen Nachrichtenversand sowie eine Datenserialisierung erfordert [7].

2.3.3 Dart Foreign Function Interface (FFI)

Dart FFI ermöglicht es, Funktionen aus nativen Bibliotheken direkt aus Dart heraus aufzurufen. Im Gegensatz zu Platform Channels erfolgt die Kommunikation hierbei nicht über Nachrichtenkanäle, sondern über direkte Funktionsaufrufe [8] [11].

Das grundlegende Prinzip eines Foreign Function Interface besteht darin, dass eine Programmiersprache Funktionen aus einer anderen Programmiersprache aufrufen kann. Dart FFI erlaubt insbesondere den Zugriff auf C-kompatible Bibliotheken. Viele Programmiersprachen, darunter C++, Rust oder Swift, können entsprechende C-kompatible Schnittstellen bereitstellen und dadurch über FFI angesprochen werden [8] [2].

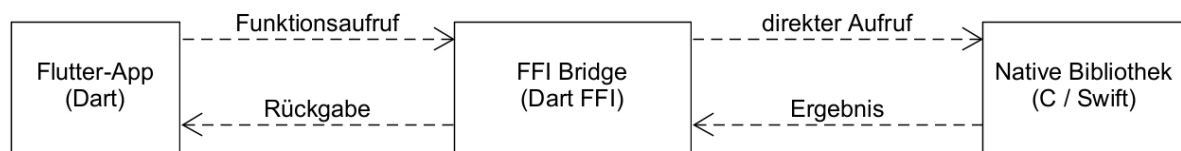


Abbildung 2-12: Schematischer Ablauf eines direkten Funktionsaufrufs über Dart FFI

Abbildung 2-12 zeigt den direkten Kommunikationsablauf über Dart FFI. Im Gegensatz zu Platform Channels erfolgt die Kommunikation nicht über eine nachrichtenbasierte Zwischenschicht, sondern über direkte Funktionsaufrufe zwischen der Dart-Laufzeitumgebung und der nativen Bibliothek. Dadurch entfällt die Serialisierung der Daten, was zu einer effizienteren Ausführung führt.

Ein vereinfachtes Beispiel für die Nutzung von Dart FFI ist in Abbildung 2-13 und Abbildung 2-14 dargestellt.

```
final dylib = DynamicLibrary.open("libnative.so");

final add = dylib.lookupFunction<
  Int32 Function(Int32, Int32),
  int Function(int, int)
>("add_numbers");

int result = add(3, 5);
```

Abbildung 2-13: Aufruf einer nativen Funktion über Dart FFI

```
int32_t add_numbers(int32_t a, int32_t b) {
    return a + b;
}
```

Abbildung 2-14: C-kompatible native Funktion

In diesem Beispiel wird eine native Funktion direkt aus einer dynamischen Bibliothek geladen und ohne zusätzliche Kommunikationsschicht aufgerufen. Der Funktionsaufruf erfolgt synchron und ohne Serialisierung, wodurch ein minimaler Overhead entsteht.

Der Integrationsprozess umfasst mehrere Schritte. Zunächst wird eine native Bibliothek erstellt, die eine C-kompatible Schnittstelle bereitstellt. Diese Bibliothek wird anschließend zur Laufzeit von der Dart-Anwendung geladen. Die darin enthaltenen Funktionen können danach direkt über definierte Funktionssignaturen aufgerufen werden [8].

Da Funktionsaufrufe über FFI ohne Nachrichtenserialisierung erfolgen, entsteht ein deutlich geringerer Kommunikationsaufwand. Dadurch eignet sich dieser Ansatz besonders für rechenintensive Operationen oder kontinuierliche Datenverarbeitung, beispielsweise bei Audioverarbeitung oder KI-gestützter Textanalyse [7] [8].

Trotz der hohen Performance ist der Einsatz von Dart FFI nicht in allen Anwendungsszenarien sinnvoll. Ein zentraler Grund liegt in der erhöhten Komplexität und den damit verbundenen Risiken. Im Gegensatz zu Platform Channels müssen bei der Verwendung von FFI Aspekte wie Speicherverwaltung, Pointer-Sicherheit und Datentypkompatibilität manuell berücksichtigt werden. Fehler in diesen Bereichen können zu schwer nachvollziehbaren Laufzeitfehlern oder

Abstürzen führen [8]. Darüber hinaus ist die Entwicklung plattformabhängiger, da native Bibliotheken für jede Zielplattform separat erstellt und eingebunden werden müssen. Dies erhöht den Wartungsaufwand und erschwert die Portierbarkeit von Anwendungen [11]. Ein weiterer Nachteil besteht darin, dass Dart FFI keinen direkten Zugriff auf plattformspezifische System-APIs wie Kamera- oder Sensorschnittstellen bietet. Solche Funktionen können zwar grundsätzlich über zusätzliche native Schnittstellen eingebunden werden, jedoch ist dies mit höherem Implementierungsaufwand verbunden. In diesen Fällen sind Platform Channels in der Praxis häufig besser geeignet [11].

Ein wichtiger Aspekt bei der Verwendung von FFI ist die Speicherverwaltung. Da Dart und native Sprachen unterschiedliche Speicherverwaltungsmodelle verwenden, muss der Speicherzugriff sorgfältig kontrolliert werden. Typische Herausforderungen betreffen insbesondere Pointer-Sicherheit, UTF-8-Konvertierung von Zeichenketten sowie die Lebensdauer nativer Objekte [8].

2.3.4 Vergleich der Integrationsansätze

Die Unterschiede zwischen den beiden Integrationsansätzen werden in Tabelle 2-3 vergleichend dargestellt.

Eigenschaft	Platform Channel	FFI
Funktionsaufruf	Nachrichtenbasiert	Direkter Funktionsaufruf
Serialisierung	Erforderlich	Nicht erforderlich
Latenz	höher	geringer
Geeignet für	API-Aufrufe	Performanzkritische Logik

Tabelle 2-3: Vergleich von Platform Channels und FFI

Der Vergleich in Tabelle 2-3 verdeutlicht, während Platform Channels vor allem für die Integration einzelner Systemfunktionen geeignet sind, bietet FFI Vorteile bei performanzkritischen Anwendungen. Da Funktionsaufrufe direkt erfolgen, können Serialisierungskosten sowie Kontextwechsel reduziert werden.

2.3.5 Relevanz für diese Arbeit

Für die in dieser Arbeit entwickelte Anwendung ist eine performante Kommunikation zwischen Flutter und nativer Logik erforderlich. Insbesondere die Verarbeitung von Sprachdaten sowie die KI-gestützte Analyse von Textinhalten stellen rechenintensive Aufgaben dar, die eine effiziente Integration nativer Komponenten erfordern.

Aus diesem Grund wird in der vorliegenden Arbeit das Foreign Function Interface verwendet, um eine direkte Verbindung zwischen der Flutter-Anwendung und einer nativen Swift-Komponente herzustellen. Dieser Ansatz ermöglicht eine klare Trennung zwischen Benutzeroberfläche und Verarbeitungslogik sowie eine performante Ausführung systemnaher Funktionen.

2.4 Sprachverarbeitung

Sprachverarbeitung bezeichnet die automatische Analyse und Verarbeitung menschlicher Sprache durch Computersysteme [17]. Sie stellt einen zentralen Bestandteil moderner Assistenzsysteme und des Natural Language Processing (NLP) dar. Natural Language Processing bezeichnet ein Teilgebiet der künstlichen Intelligenz, das sich mit der automatischen Verarbeitung und Analyse natürlicher Sprache befasst [18] [17]. Dadurch wird eine natürliche Interaktion zwischen Mensch und Maschine ermöglicht [17].

In mobilen Anwendungen wird Sprachverarbeitung häufig eingesetzt, um Eingaben über gesprochene Sprache zu ermöglichen oder Informationen akustisch auszugeben [18].

Im Kontext dieser Arbeit spielt Sprachverarbeitung eine wichtige Rolle, da Gesprächsinhalte nach einem Kundengespräch schnell und möglichst natürlich erfasst werden sollen. Hierzu werden Verfahren der automatischen Spracherkennung (STT) sowie der Sprachsynthese (TTS) eingesetzt.

2.4.1 Speech-to-Text (STT)

STT bezeichnet die automatische Umwandlung gesprochener Sprache in geschriebenen Text. Ziel solcher Systeme ist es, akustische Signale eines Mikrofons zu analysieren und daraus sprachliche Inhalte zu rekonstruieren [17].

Moderne Spracherkennungssysteme bestehen typischerweise aus mehreren Verarbeitungsschritten. Zunächst wird das Audiosignal über das Mikrofon aufgenommen und digitalisiert. Anschließend erfolgt eine Merkmalsextraktion, bei der charakteristische Eigenschaften des Audiosignals bestimmt werden. Diese Merkmale werden anschließend von einem akustischen Modell verarbeitet, das die Beziehung zwischen Audiosignalen und sprachlichen Einheiten beschreibt [17].

Zusätzlich kommt ein Sprachmodell zum Einsatz, das die Wahrscheinlichkeit bestimmter Wortfolgen bewertet [17]. Durch die Kombination aus akustischem Modell und Sprachmodell kann das System entscheiden, welche Wortsequenz am wahrscheinlichsten zu dem aufgenommenen Audiosignal passt. Abschließend erzeugt ein Dekodierungsalgorithmus die finale Textausgabe.

Ein vereinfachter Ablauf eines STT-Systems lässt sich wie folgt beschreiben:

1. Aufnahme des Audiosignals
2. Merkmalsextraktion aus dem Audiosignal
3. Analyse durch ein akustisches Modell
4. Bewertung möglicher Wortfolgen durch ein Sprachmodell
5. Generierung des erkannten Textes

STT-Systeme können entweder cloudbasiert oder lokal auf dem Endgerät betrieben werden. Cloudbasierte Lösungen bieten häufig eine hohe Erkennungsgenauigkeit, erfordern jedoch eine Internetverbindung. Offline-Systeme führen die Verarbeitung vollständig auf dem Gerät aus und bieten dadurch Vorteile hinsichtlich Datenschutz und Systemverfügbarkeit.

Für mobile Anwendungen, die auch ohne Internetverbindung funktionieren sollen, ist die lokale Verarbeitung besonders relevant. Dadurch können Sprachdaten direkt auf dem Endgerät verarbeitet werden, ohne dass sensible Informationen an externe Server übertragen werden müssen [18] [17].

2.4.2 Text-to-Speech (TTS)

TTS bezeichnet die synthetische Erzeugung gesprochener Sprache aus geschriebenem Text. Ziel solcher Systeme ist es, textuelle Informationen in ein natürlich klingendes Audiosignal umzuwandeln und dadurch eine akustische Interaktion zwischen Mensch und Computersystem zu ermöglichen [17].

Der Prozess der Sprachsynthese umfasst mehrere Verarbeitungsschritte. Zunächst wird der eingegebene Text linguistisch analysiert, um Satzstruktur, Betonung und Aussprache zu bestimmen. Anschließend wird der Text in phonemische Einheiten umgewandelt, die die grundlegenden Laute einer Sprache repräsentieren. Diese phonemischen Informationen werden anschließend von einem Sprachsynthese-Modell in ein Audiosignal umgewandelt [17].

Moderne TTS-Systeme verwenden unterschiedliche Syntheseverfahren. Frühere Ansätze basierten auf der Kombination voraufgenommener Sprachsegmente, während moderne Systeme zunehmend neuronale Modelle einsetzen, um natürlicher klingende Sprache zu erzeugen [18].

In mobilen Anwendungen wird TTS häufig eingesetzt, um Rückmeldungen oder generierte Inhalte akustisch auszugeben. Dadurch kann die Benutzerinteraktion verbessert werden, insbesondere in Situationen, in denen eine visuelle Interaktion mit der Anwendung eingeschränkt ist.

2.4.3 Relevanz für diese Arbeit

Für den in dieser Arbeit entwickelten Kontaktbericht-Assistenten spielt insbesondere die STT-Technologie eine zentrale Rolle. Sie ermöglicht es, Gesprächsinhalte unmittelbar nach einem Kundengespräch per Spracheingabe zu erfassen. Dadurch können Nutzerinnen und Nutzer Informationen schneller und natürlicher eingeben als über eine manuelle Texteingabe.

Zusätzlich kann TTS genutzt werden, um Rückmeldungen oder generierte Inhalte akustisch auszugeben. Dies unterstützt eine benutzerfreundliche Interaktion mit der Anwendung und erweitert die Möglichkeiten der sprachbasierten Bedienung.

Die in diesem Kapitel beschriebenen Konzepte der Sprachverarbeitung bilden die theoretische Grundlage für die Implementierung der sprachbasierten Interaktion im Kontaktbericht-Assistenten. In der praktischen Umsetzung werden diese Konzepte mithilfe von durch Apple bereitgestellten Frameworks realisiert. Für die Spracherkennung wird unter iOS das Speech Framework verwendet, während die Audioaufnahme über die Komponente AVAudioEngine erfolgt. Die Sprachausgabe wird über das AVFoundation Framework und den AVSpeechSynthesizer umgesetzt. Diese nativen Funktionen werden innerhalb einer separaten Swift-Komponente implementiert und anschließend über eine FFI-basierte Schnittstelle in die Flutter-Anwendung integriert. Die konkrete technische Umsetzung dieser Architektur sowie die Implementierung der einzelnen Komponenten werden in Kapitel 5 detailliert beschrieben.

3 Anforderungen an den KI-basierten Kontaktbericht-Assistenten

Ziel dieses Kapitels ist die systematische Definition der funktionalen und nicht-funktionalen Anforderungen an den im Rahmen dieser Arbeit entwickelten offlinefähigen KI-basierten Kontaktbericht-Assistenten. Die Anforderungen bilden die Grundlage für die späteren Architektur- und Implementierungsentscheidungen und leiten sich aus den spezifischen Anforderungen des Vertriebsalltags sowie den technischen Rahmenbedingungen mobiler Anwendungen ab.

Im Vertrieb werden Kontaktberichte häufig unmittelbar nach Kundengesprächen erstellt. Dieser Prozess ist in vielen Fällen zeitaufwendig, da Gesprächsinhalte zunächst frei formuliert erfasst und anschließend manuell in eine strukturierte Berichtsform überführt werden müssen. Ziel des entwickelten Systems ist es daher, Vertriebsmitarbeiter bei der effizienten Erstellung solcher Berichte zu unterstützen. Der Assistent soll gesprochene oder frei formulierte Inhalte automatisch analysieren und relevante Informationen in eine standardisierte Berichtsstruktur überführen.

3.1 Funktionale Anforderungen

Die funktionalen Anforderungen beschreiben die konkreten Systemfunktionen und Fähigkeiten, die der Kontaktbericht-Assistent bereitstellen muss, um die beschriebenen Ziele zu erreichen.

3.1.1 Erfassung von Gesprächsinhalten

Das System muss die Erfassung von Gesprächsinhalten über verschiedene Eingabemethoden ermöglichen. Dazu gehören sowohl die Spracheingabe über das mobile Endgerät als auch die manuelle Eingabe von Freitext durch den Benutzer.

Bei der Spracheingabe wird das gesprochene Audio zunächst aufgezeichnet und anschließend automatisch in Text transkribiert. Diese Verarbeitung muss vollständig lokal auf dem Endgerät erfolgen, sodass keine Internetverbindung erforderlich ist. Der erkannte Text dient anschließend als Grundlage für die weitere Analyse und Strukturierung des Kontaktberichts.

Neben der Spracheingabe muss das System auch die direkte Texteingabe unterstützen, um eine flexible Nutzung in unterschiedlichen Situationen zu ermöglichen.

3.1.2 KI-gestützte Strukturierung von Freitext

Der Kontaktbericht-Assistent muss in der Lage sein, unstrukturierte Texte automatisch zu analysieren und daraus relevante Informationen zu extrahieren. Ziel dieser Verarbeitung ist die Überführung frei formulierter Gesprächsinhalte in eine strukturierte Form, die für Kontaktberichte verwendet werden kann.

Zu den relevanten Informationen zählen insbesondere das Datum des Gesprächs, die Art des Kontakts (beispielsweise Telefonat oder Messegespräch), der beteiligte Ansprechpartner sowie der wesentliche Inhalt des Gesprächs. Diese Informationen sollen automatisch erkannt und in entsprechende Felder eines strukturierten Kontaktberichts überführt werden.

Die extrahierten Informationen müssen anschließend in einer Form bereitgestellt werden, die eine weitere Bearbeitung und Speicherung innerhalb der Anwendung ermöglicht.

3.1.3 Offline-Fähigkeit der Verarbeitung

Ein wesentliches funktionales Ziel des Systems ist die vollständige Offline-Fähigkeit der Verarbeitung. Sämtliche KI-gestützten Analyseprozesse müssen lokal auf dem mobilen Endgerät durchgeführt werden. Es dürfen keine externen Server oder cloudbasierte Dienste zur Verarbeitung der Sprach- oder Textdaten verwendet werden.

Diese Anforderung dient einerseits der Sicherstellung der Einsatzfähigkeit in Umgebungen mit eingeschränkter oder fehlender Internetverbindung. Andererseits trägt sie zum Schutz sensibler Kundendaten bei, da keine Übertragung personenbezogener Informationen an externe Systeme erfolgt.

3.1.4 Nachbearbeitung durch den Nutzer

Da automatische Extraktionsverfahren nicht in allen Fällen vollständig fehlerfrei arbeiten können, muss der Benutzer die Möglichkeit haben, die automatisch generierten Inhalte zu überprüfen und bei Bedarf zu korrigieren.

Die Anwendung muss daher Funktionen bereitstellen, mit denen automatisch erkannte Informationen manuell angepasst oder ergänzt werden können. Darüber hinaus muss der Benutzer fehlende Angaben hinzufügen können, bevor der Kontaktbericht gespeichert wird. Automatische Änderungen dürfen nicht irreversibel erfolgen, sodass der Benutzer jederzeit die Kontrolle über den finalen Inhalt des Berichts behält.

3.1.5 Lokale Speicherung von Kontaktberichten

Erstellte Kontaktberichte müssen dauerhaft auf dem Gerät gespeichert werden können. Das System muss daher Funktionen zum Speichern neuer Berichte, zum Aktualisieren bestehender Berichte sowie zum Laden bereits gespeicherter Einträge bereitstellen.

Die Speicherung erfolgt ausschließlich lokal innerhalb der Anwendung. Eine externe Datenübertragung oder Synchronisation mit Cloud-Diensten ist nicht vorgesehen.

3.1.6 Zusammenfassung der funktionalen Anforderungen

Die zuvor beschriebenen funktionalen Anforderungen werden im Folgenden zusammenfassend dargestellt. Die strukturierte Übersicht dient der kompakten Darstellung der zentralen Systemfunktionen sowie deren Priorisierung im Rahmen der prototypischen Umsetzung.

ID	Anforderung	Beschreibung	Priorität
F1	Spracheingabe	Gesprächsinhalte sollen per Sprache aufgenommen werden können	Muss
F2	Texteingabe	Nutzer können Gesprächsinhalte auch manuell eingeben	Muss
F3	KI-Analyse	Freitext soll automatisch analysiert werden	Muss
F4	Strukturierter Bericht	Extrahierte Informationen sollen strukturiert dargestellt werden	Muss
F5	Nachbearbeitung	Nutzer können automatisch extrahierte Inhalte korrigieren	Soll
F6	Lokale Speicherung	Kontaktberichte müssen gespeichert und geladen werden können	Muss

Tabelle 3-1: Übersicht der funktionalen Anforderungen

Tabelle 3-1 fasst die wesentlichen funktionalen Anforderungen des Systems zusammen. Die Anforderungen decken sowohl die Erfassung von Gesprächsinhalten als auch deren KI-gestützte Verarbeitung und strukturierte Aufbereitung ab.

Die Priorisierung erfolgt anhand der Kategorien „Muss-Anforderungen“ und „Soll-Anforderungen“.

Muss-Anforderungen sind zwingend für die grundlegende Funktionsfähigkeit des Systems erforderlich, während Soll-Anforderungen optionale Erweiterungen darstellen, die die Benutzerfreundlichkeit und Flexibilität erhöhen.

3.2 Nicht-funktionale Anforderungen

Neben den funktionalen Anforderungen spielen auch nicht-funktionale Anforderungen eine wichtige Rolle. Diese beschreiben qualitative Eigenschaften des Systems, wie beispielsweise Performance, Sicherheit oder Benutzerfreundlichkeit.

3.2.1 Datenschutz und Datensicherheit

Da Kontaktberichte personenbezogene sowie unternehmensrelevante Informationen enthalten können, müssen besondere Anforderungen an Datenschutz und Datensicherheit erfüllt werden. Alle sensiblen Daten müssen vollständig lokal verarbeitet werden, ohne dass eine Übertragung an externe Server erfolgt.

Die Anwendung darf keine cloudbasierten KI-Dienste verwenden und muss sicherstellen, dass alle Sprach- und Textdaten ausschließlich innerhalb der lokalen App-Umgebung verarbeitet und gespeichert werden.

3.2.2 Performance

Die Anwendung muss auf aktuellen mobilen iOS-Endgeräten, wie beispielsweise einem iPhone 15 Pro oder neuer, flüssig und reaktionsschnell ausführbar sein. Die Verarbeitung von Sprach-eingaben sowie die Analyse von Textdaten soll innerhalb von maximal 2 bis 3 Sekunden erfolgen, um eine effiziente Nutzung im Arbeitsalltag zu gewährleisten.

Während der Verarbeitung darf die Benutzeroberfläche nicht blockieren, sodass der Benutzer weiterhin mit der Anwendung interagieren kann. Rechenintensive Operationen sind daher asynchron auszuführen. Zudem müssen Speichervorgänge sowie abgeschlossene Verarbeitungsschritte unmittelbar visuell rückgemeldet werden, beispielsweise durch Statusanzeigen oder Ladeindikatoren.

Eine geringe Latenz ist insbesondere im Vertriebsumfeld von hoher Bedeutung, da Kontaktberichte häufig unmittelbar nach Kundengesprächen erstellt werden und eine verzögerte Verarbeitung den Arbeitsfluss negativ beeinflussen kann.

3.2.3 Benutzerfreundlichkeit

Die Anwendung muss eine benutzerfreundliche und effizient bedienbare Benutzeroberfläche bereitstellen. Die Erstellung eines Kontaktberichts soll mit maximal drei Interaktionen (z. B. Start der Aufnahme, Bestätigung der Extraktion, Speichern) möglich sein. Die Benutzeroberfläche muss klar strukturiert sein, sodass alle zentralen Funktionen (Aufnahme, Anzeige, Bearbeitung und Speicherung) ohne Navigation durch mehr als zwei Ebenen erreichbar sind.

Während der Verarbeitung müssen dem Benutzer innerhalb von höchstens 1 Sekunde visuelle Rückmeldungen angezeigt werden, beispielsweise in Form von Ladeindikatoren oder Statusmeldungen. Automatisch extrahierte Informationen müssen vollständig und unmittelbar nach der Verarbeitung sichtbar sein. Änderungen durch den Benutzer müssen direkt im Interface angezeigt werden, ohne dass ein erneutes Laden erforderlich ist.

Darüber hinaus muss der Benutzer in der Lage sein, alle automatisch erkannten Felder manuell zu korrigieren oder zu ergänzen. Für die Bearbeitung eines Kontaktberichts dürfen maximal zwei Eingabefelder erforderlich sein.

3.2.4 Wartbarkeit und Erweiterbarkeit

Die Systemarchitektur muss so gestaltet sein, dass zukünftige Erweiterungen oder Anpassungen möglich sind. Insbesondere soll es möglich sein, eingesetzte KI-Modelle auszutauschen oder die Extraktionslogik zu erweitern.

Darüber hinaus sollen einzelne Systemkomponenten unabhängig voneinander weiterentwickelt werden können. Eine modulare Architektur ist daher eine wichtige Voraussetzung für die langfristige Wartbarkeit und Erweiterbarkeit des Systems.

3.2.5 Plattformintegration

Die Anwendung muss sich in die technischen Rahmenbedingungen mobiler iOS-Endgeräte integrieren lassen. Dazu gehört insbesondere die Nutzung nativer Audio-Schnittstellen für die Sprachaufnahme sowie der Zugriff auf Mikrofonberechtigungen.

Darüber hinaus muss die Anwendung ressourcenschonend arbeiten, um eine stabile Nutzung auf mobilen Geräten zu gewährleisten. Die Architektur muss daher eine effiziente Zusammenarbeit zwischen der plattformübergreifenden Benutzeroberfläche und der nativen Verarbeitungslogik ermöglichen.

3.2.6 Zusammenfassung der nicht-funktionalen Anforderungen

Die zuvor beschriebenen nicht-funktionalen Anforderungen werden im Folgenden zusammenfassend dargestellt. Die Übersicht dient der strukturierten Darstellung der qualitativen Eigenschaften des Systems sowie deren Priorisierung im Hinblick auf die prototypische Umsetzung.

ID	Anforderung	Beschreibung	Priorität
NF1	Offline-Fähigkeit	System muss ohne Internetverbindung funktionieren	Muss
NF2	Datenschutz	Sensible Daten dürfen nicht an externe Server übertragen werden	Muss
NF3	Performance	Analyse und Verarbeitung sollen innerhalb von maximal 2 bis 3 Sekunden abgeschlossen sein	Soll
NF4	Benutzerfreundlichkeit	Ein Kontaktbericht muss mit maximal drei Benutzerinteraktionen erstellt werden können und visuelle Rückmeldungen müssen innerhalb von maximal 1 Sekunde erfolgen.	Soll
NF5	Wartbarkeit	Systemarchitektur soll modular aufgebaut sein	Soll
NF6	Plattform-integration	Zugriff auf Mikrofon und Systemfunktionen muss möglich sein	Muss

Tabelle 3-2: Übersicht der nicht-funktionalen Anforderungen

Tabelle 3-2 zeigt die zentralen nicht-funktionalen Anforderungen des Systems. Besonders hervorzuheben sind die Anforderungen an die Offline-Fähigkeit und den Datenschutz, die als Muss-Anforderungen definiert wurden und somit essenziell für den Einsatz im Vertriebskontext sind.

Darüber hinaus spielen Performance und Benutzerfreundlichkeit eine wichtige Rolle für die praktische Nutzbarkeit der Anwendung im Arbeitsalltag. Anforderungen wie Wartbarkeit und Plattformintegration stellen sicher, dass das System langfristig erweiterbar und stabil betreibbar ist.

Die nicht-funktionalen Anforderungen ergänzen die funktionalen Anforderungen und bilden gemeinsam die Grundlage für die Systemarchitektur sowie die prototypische Umsetzung.

3.3 Technische Rahmenbedingungen

Die Entwicklung des Systems erfolgt auf Basis einer Cross-Platform-Architektur mit Flutter als Technologie für die Benutzeroberfläche. Bestimmte Funktionen, insbesondere die Audioverarbeitung sowie Teile der KI-gestützten Analyse, erfordern jedoch systemnahe Zugriffe auf native Plattformfunktionen.

Daraus ergibt sich die Notwendigkeit, native Komponenten in die Anwendung zu integrieren. Die Architektur muss daher eine klare Trennung zwischen der Benutzeroberfläche und der Verarbeitungslogik gewährleisten. Während die Benutzerinteraktion in Flutter implementiert wird, erfolgt die Verarbeitung der Audio- und Textdaten in einer nativen Komponente.

Darüber hinaus muss ein geeigneter Schnittstellenmechanismus zwischen Dart und der nativen Implementierung bereitgestellt werden, um eine effiziente Kommunikation zwischen beiden Systemteilen zu ermöglichen. Die konkrete technische Umsetzung dieser Architektur wird in Kapitel 4 detailliert beschrieben.

3.4 Zusammenfassung der Anforderungen

Die definierten Anforderungen bilden die Grundlage für die im folgenden Kapitel beschriebene Systemarchitektur sowie die anschließende prototypische Umsetzung.

In Kapitel 4 wird die Architektur des Systems detailliert dargestellt. Anschließend wird in Kapitel 5 die konkrete Implementierung beschrieben und erläutert, wie die einzelnen Anforderungen technisch realisiert wurden.

4 Systemarchitektur

In diesem Kapitel wird die Systemarchitektur des entwickelten Prototyps beschrieben. Ziel der Architektur ist es, eine modulare, performante und datenschutzkonforme Umsetzung eines offlinefähigen KI-basierten Kontaktbericht-Assistenten zu ermöglichen.

4.1 Architekturüberblick

Abbildung 4-1 zeigt die grundlegende Systemarchitektur des entwickelten Kontaktbericht-Assistenten. Die Architektur folgt einem mehrschichtigen Ansatz, bei dem Benutzeroberfläche, Integrationsschicht und native Verarbeitungslogik klar voneinander getrennt sind.

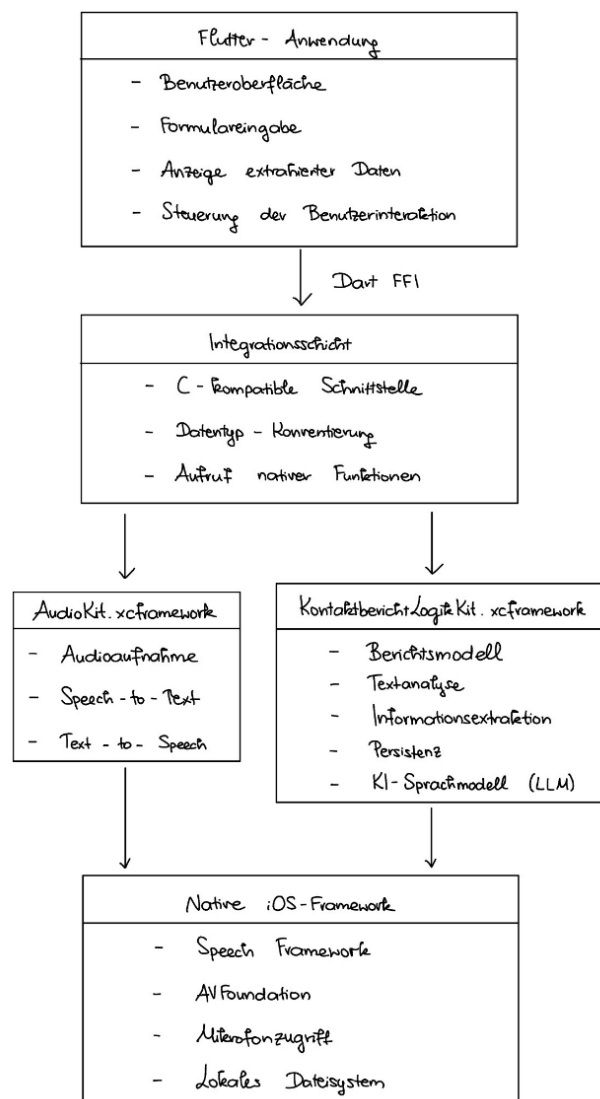


Abbildung 4-1: Gesamtarchitektur des Kontaktbericht-Assistenten

Die Architektur ermöglicht somit eine klare Trennung zwischen Präsentation, Integrationslogik und fachlicher Verarbeitung. Dadurch wird eine hohe Wartbarkeit sowie eine flexible Erweiterbarkeit des Systems erreicht.

Tabelle 4-1 zeigt eine strukturierte Übersicht der zentralen Architekturkomponenten, ihrer technologischen Umsetzung sowie ihrer jeweiligen Aufgaben innerhalb des Systems.

Komponente	Technologie	Hauptaufgabe	Funktionen
Benutzeroberfläche	Flutter / Dart	Interaktion mit dem Benutzer, Darstellung und Bearbeitung von Kontaktberichten	Formulareingabe, Anzeige extrahierter Daten, Benutzersteuerung
Integrationsschicht	Dart FFI	Verbindung zwischen Flutter und nativer Logik	Aufruf nativer Funktionen, Datentyp-Konvertierung
Native Verarbeitungs-module	Swift	Sprachverarbeitung, Textanalyse und Persistenz	STT, TTS, Informationsextraktion, lokale Speicherung
Framework-Bereitstellung	XCFramework	Bereitstellung der nativen Swift-Komponente für verschiedene Zielarchitekturen	Einbindung in iOS-Projekt, Wiederverwendbarkeit
Systemnahe Dienste	IOS-Frameworks	Zugriff auf Betriebssystem-funktionen	Speech Framework, AVFoundation, Mikrofonzugriff, Dateisystem

Tabelle 4-1: Überblick über die Architekturkomponenten des Systems

Der Vergleich verdeutlicht, dass die einzelnen Komponenten klar abgegrenzte Verantwortlichkeiten besitzen. Während die Benutzeroberfläche für die Interaktion zuständig ist, übernimmt

die Integrationsschicht die technische Kommunikation, und die native Verarbeitungsschicht realisiert die eigentliche Geschäftslogik. Diese Aufteilung trägt maßgeblich zur Modularität und Skalierbarkeit der Systemarchitektur bei.

4.2 Architekturstil und Entwurfsprinzipien

Die Architektur des entwickelten Kontaktbericht-Assistenten orientiert sich an mehreren grundlegenden Entwurfsprinzipien der Softwareentwicklung. Ziel dieser Prinzipien ist es, eine modulare, wartbare und erweiterbare Systemstruktur zu schaffen, die gleichzeitig eine effiziente Verarbeitung von Daten ermöglicht.

Ein zentrales Prinzip der Systemarchitektur ist die Trennung unterschiedlicher Verantwortlichkeiten (Separation of Concerns), die eine klare Strukturierung von Softwarekomponenten ermöglicht und die Wartbarkeit erhöht [19]. Die Benutzeroberfläche, die Integrationsschicht sowie die eigentliche Verarbeitungslogik sind in klar voneinander getrennte Komponenten unterteilt. Die Flutter-Anwendung übernimmt ausschließlich die Darstellung der Benutzeroberfläche sowie die Interaktion mit dem Benutzer. Die eigentliche Verarbeitung von Sprach- und Textdaten erfolgt hingegen innerhalb nativer Swift-Bibliotheken. Durch diese Aufteilung können Änderungen an der Benutzeroberfläche unabhängig von der Verarbeitungslogik vorgenommen werden.

Ein weiteres wichtiges Prinzip ist die Modularisierung der Systemkomponenten, welche die Reduktion von Abhängigkeiten sowie die Wiederverwendbarkeit einzelner Komponenten unterstützt [20]. Die native Verarbeitungslogik ist in zwei eigenständige Module unterteilt: das Modul AudioKit für die Audioverarbeitung sowie das Modul KontaktberichtLogikKit für die Analyse und Verwaltung von Kontaktberichten. Diese modulare Struktur reduziert Abhängigkeiten zwischen den einzelnen Systemkomponenten und erleichtert sowohl Wartung als auch Erweiterung des Systems. Darüber hinaus ermöglicht die Aufteilung eine potenzielle Wiederverwendung der einzelnen Module in anderen Anwendungen.

Darüber hinaus folgt die Architektur einem sogenannten Offline-First-Ansatz, bei dem zentrale Funktionen unabhängig von Netzwerkverbindungen ausgeführt werden [21]. Sämtliche Verarbeitungsschritte, einschließlich der Sprachtranskription sowie der Analyse von Gesprächsinhalten, erfolgen lokal auf dem mobilen Endgerät. Dadurch wird sowohl die Verfügbarkeit der Anwendung als auch der Schutz sensibler Kundendaten gewährleistet.

Ein weiteres zentrales Entwurfsprinzip ist die klare Schnittstellendefinition zwischen plattformübergreifendem und nativem Code. Die Kommunikation zwischen der Flutter-Anwendung und den nativen Swift-Modulen erfolgt über das Dart Foreign Function Interface (FFI). Durch die Verwendung C-kompatibler Funktionssignaturen wird eine lose Kopplung zwischen den Systemkomponenten erreicht, wodurch Wartbarkeit und Erweiterbarkeit verbessert werden [20].

Die Schnittstellen zwischen den einzelnen Architekturschichten sind in Abbildung 5-1 dargestellt.

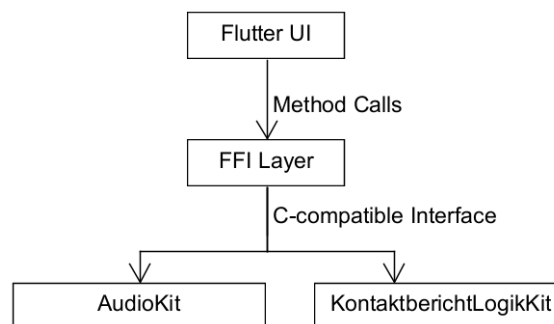


Abbildung 4-2: Schnittstellenmodell zwischen Flutter-Anwendung und nativen Swift-Modulen

Abbildung 4-2 zeigt die definierte Schnittstelle zwischen der Flutter-Anwendung und den nativen Swift-Modulen. Methodenaufrufe werden von der Flutter-Schicht über die FFI-Schicht an die nativen Module weitergeleitet, während Verarbeitungsergebnisse in umgekehrter Richtung zurückgegeben werden. Der Datenaustausch erfolgt über klar definierte Funktionssignaturen sowie kompatible Datentypen. Diese klare Schnittstellendefinition reduziert die Kopplung zwischen den Komponenten und erhöht die Wartbarkeit des Systems.

4.3 Native Verarbeitungslogik

Die zentrale Verarbeitungslogik des Kontaktbericht-Assistenten ist in nativen Swift-Bibliotheken implementiert. Diese Module übernehmen Aufgaben, die einen direkten Zugriff auf systemnahe Funktionen oder eine besonders effiziente Verarbeitung erfordern. Die native Implementierung ermöglicht dabei eine performante Nutzung der iOS-Systemframeworks sowie eine effiziente Verarbeitung von Audio- und Textdaten.

Die native Logik ist in zwei separate Module unterteilt: AudioKit und KontaktberichtLogikKit. Beide Module sind als eigenständige Bibliotheken implementiert und werden über eine FFI-basierte Schnittstelle in die Flutter-Anwendung integriert.

4.3.1 AudioKit

Das Modul AudioKit ist für die Verarbeitung von Spracheingaben sowie für die Sprachausgabe verantwortlich. Der interne Aufbau des Moduls folgt einer mehrstufigen Struktur aus Fassade, Orchestrierung, spezialisierten Verarbeitungsdiensten und FFI-Schnittstelle.

Die Komponentenstruktur des Moduls AudioKit ist in Abbildung 4-3 dargestellt.

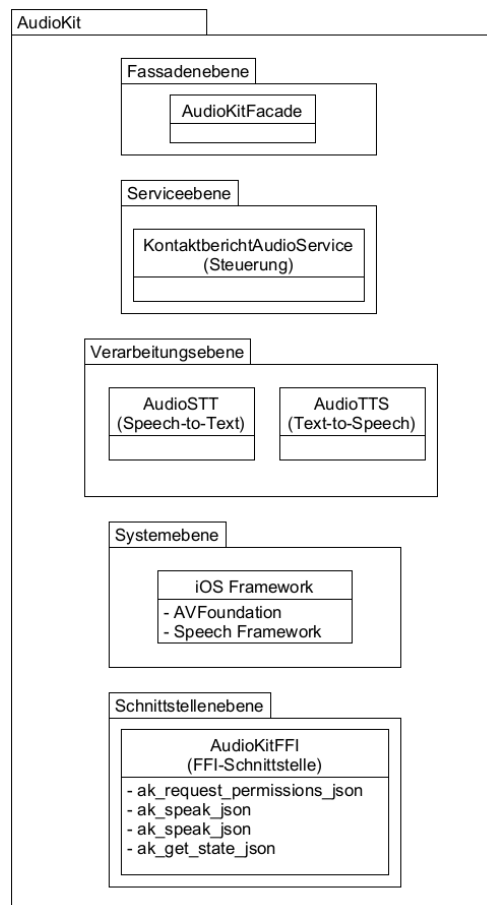


Abbildung 4-3: Interner Aufbau des nativen Moduls AudioKit

Abbildung 4-3 zeigt den schichtenartigen Aufbau des Moduls AudioKit. Die Fassadenebene kapselt den Zugriff auf die Audiofunktionen nach außen, während die Serviceebene die Ab-

laufsteuerung übernimmt. Die eigentliche Audioverarbeitung wird durch die Komponenten AudioSTT und AudioTTS realisiert, die wiederum auf systemnahe Frameworks wie AVFoundation und das Speech Framework zugreifen. Die Anbindung an die Flutter-Anwendung erfolgt über die FFI-Schnittstellenebene.

4.3.2 KontaktberichtLogikKit

Das Modul KontaktberichtLogikKit implementiert die fachliche Verarbeitungslogik des Kontaktbericht-Assistenten. Es ist verantwortlich für die Analyse unstrukturierter Gesprächsinhalte, die Überführung in strukturierte Kontaktberichte sowie für deren lokale Speicherung. Der interne Aufbau des Moduls folgt einer klaren Paketstruktur, die in Modelle, Services sowie unterstützende Komponenten (Supports) unterteilt ist.

Die Komponentenstruktur des Moduls KontaktberichtLogikKit ist in Abbildung 4-4 dargestellt.

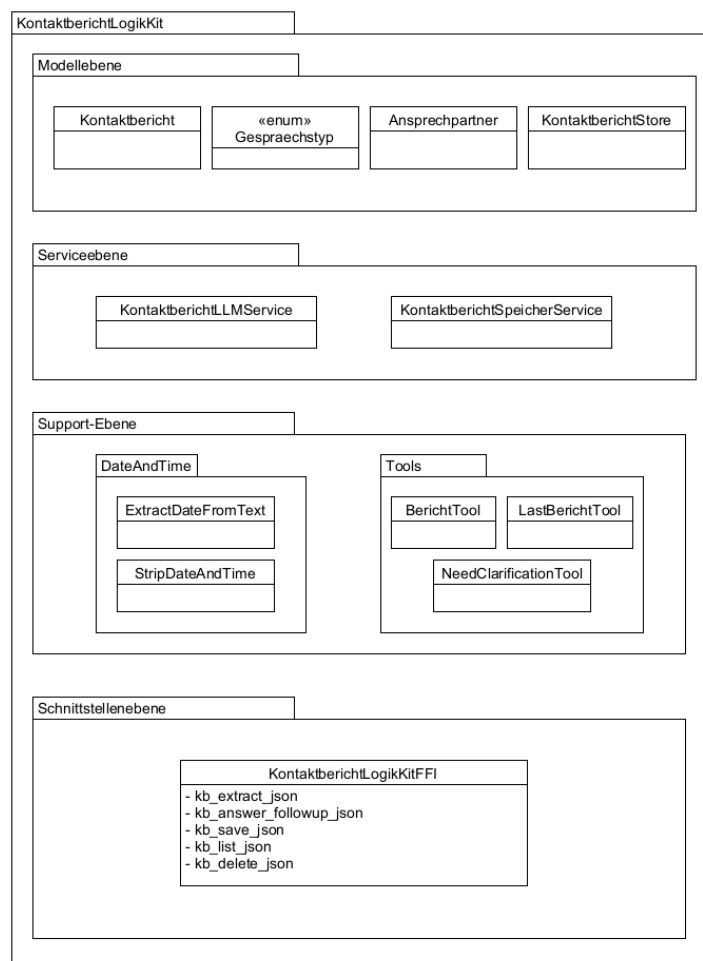


Abbildung 4-4: Paketstruktur des Moduls KontaktberichtLogikKit

Abbildung 4-4 zeigt die interne Gliederung des Moduls KontaktberichtLogikKit in Modell-, Service-, Support- und Schnittstellenebene. Während die Modellebene die fachlichen Datenstrukturen beschreibt, kapselt die Serviceebene die Analyse- und Speicherlogik. Die Support-Ebene stellt spezialisierte Hilfskomponenten für Datumsverarbeitung und Berichtserzeugung bereit. Die Schnittstellenebene definiert die FFI-basierten Einstiegspunkte für die Kommunikation mit der Flutter-Anwendung.

4.4 Kommunikation zwischen Flutter und Swift (nativen Modulen)

Die Kommunikation zwischen der Flutter-Anwendung und den nativen Swift-Komponenten erfolgt über das Dart Foreign Function Interface (FFI). Dieses Interface ermöglicht es, Funktionen aus nativen Bibliotheken direkt aus Dart heraus aufzurufen und damit eine performante Integration systemnaher Funktionalität zu realisieren.

Über eine C-kompatible Schnittstelle werden Funktionen der nativen Module AudioKit und KontaktberichtLogikKit bereitgestellt. Die Flutter-Anwendung kann diese Funktionen über die FFI-Integrationsschicht aufrufen und Daten zwischen den unterschiedlichen Laufzeitumgebungen austauschen.

Abbildung 4-6 zeigt den sequenziellen Ablauf der Verarbeitung einer Spracheingabe innerhalb des Systems.

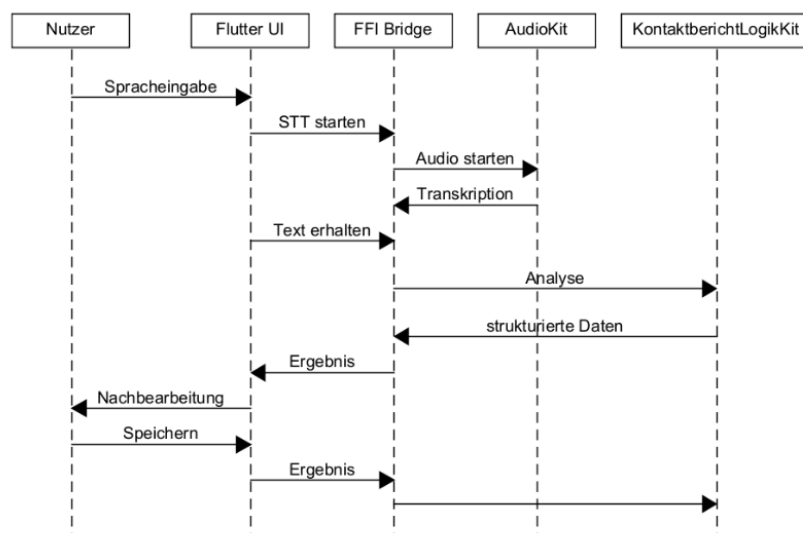


Abbildung 4-5: Sequenzdiagramm der Kommunikation zwischen Flutter-Anwendung und nativen Swift-Modulen bei der Verarbeitung einer Spracheingabe

Der Prozess beginnt mit der Initiierung einer Spracheingabe durch den Benutzer in der Flutter-basierten Benutzeroberfläche. Die Flutter-Anwendung übergibt anschließend einen Aufruf über die FFI-Bridge an das native Modul AudioKit, das für die Audioaufnahme und die STT-Verarbeitung verantwortlich ist.

Im Modul AudioKit wird zunächst die Audioaufnahme gestartet und das aufgenommene Audiosignal mithilfe des iOS Speech Frameworks in Text transkribiert. Der erzeugte Text wird anschließend an die Flutter-Anwendung zurückgegeben und über die FFI-Schnittstelle an das Modul KontaktberichtLogikKit weitergeleitet.

Das Modul KontaktberichtLogikKit führt eine Analyse des transkribierten Textes durch und extrahiert relevante Informationen für den Kontaktbericht, beispielsweise Gesprächspartner, Datum oder Gesprächsinhalte. Die extrahierten Informationen werden anschließend als strukturierte Daten an die Flutter-Anwendung zurückgegeben.

Die Flutter-Anwendung stellt diese Daten in der Benutzeroberfläche dar, sodass der Benutzer die automatisch erkannten Inhalte überprüfen und gegebenenfalls nachbearbeiten kann. Abschließend kann der Kontaktbericht gespeichert werden, wobei die Persistenzlogik erneut über die FFI-Schnittstelle im nativen Modul ausgeführt wird.

4.5 Bewertung der Architektur

Die entwickelte Architektur ist klar strukturiert und trennt Benutzeroberfläche, Integrations-schicht und native Verarbeitung logisch voneinander. Dadurch wird eine gute Wartbarkeit sowie eine performante, lokale KI-Verarbeitung ermöglicht.

Eine Einschränkung besteht darin, dass die Nutzung nativer Swift-Komponenten die Anwendung aktuell auf iOS beschränkt. Eine Erweiterung auf andere Plattformen würde zusätzliche Implementierungen erfordern. Zudem erhöht der Einsatz von FFI die technische Komplexität, insbesondere bei Schnittstellen und Datentypkonvertierungen.

Insgesamt stellt die Architektur eine geeignete Grundlage für einen offlinefähigen, KI-basierten Kontaktbericht-Assistenten dar.

5 Prototypische Umsetzung

In diesem Kapitel wird die prototypische Umsetzung des entwickelten kontaktberichts-basierten Assistenten beschrieben. Die Implementierung orientiert sich an der in Kapitel 4 dargestellten Systemarchitektur und folgt einer modularen Struktur. Die Benutzeroberfläche wurde mit Flutter umgesetzt, während systemnahe Funktionen sowie die Verarbeitungslogik in nativen Swift-Bibliotheken implementiert wurden.

Die native Funktionalität ist dabei in zwei eigenständige Module unterteilt. Das Modul AudioKit übernimmt die Audioaufnahme sowie die Sprachverarbeitung, während das Modul KontaktberichtLogikKit die Analyse von Gesprächsinhalten sowie die Verwaltung von Kontaktberichten implementiert. Beide Bibliotheken werden als XCFramework bereitgestellt und über eine FFI-basierte Integrationsschicht in die Flutter-Anwendung eingebunden.

5.1 Überblick über die Implementierung

Die Implementierung des Systems basiert auf mehreren miteinander verbundenen Komponenten. Die Flutter-Anwendung stellt die Benutzeroberfläche bereit und ermöglicht die Interaktion mit dem Benutzer. Über eine FFI-basierte Schnittstelle werden Funktionen der nativen Swift-Bibliotheken aufgerufen.

Die nativen Module übernehmen insbesondere die Audioverarbeitung, die Analyse von Gesprächsinhalten sowie die lokale Speicherung von Kontaktberichten. Dadurch wird eine effiziente Verarbeitung der Daten direkt auf dem mobilen Endgerät ermöglicht.

Die folgenden Abschnitte beschreiben die Implementierung der einzelnen Systemkomponenten im Detail.

5.2 Umsetzung der Benutzeroberfläche in Flutter

Die Benutzeroberfläche wurde mit dem Flutter-Framework umgesetzt und dient zur Eingabe, Anzeige und Bearbeitung von Kontaktberichten. Sie ermöglicht sowohl die manuelle Texteingabe als auch die Spracheingabe sowie die KI-gestützte Verarbeitung von Freitexten.

Zu Beginn kann der Nutzer entweder einen Freitext manuell eingeben oder per Sprache erfassen. Die Spracheingabe wird dabei automatisch in Text umgewandelt. Der erfasste Text kann anschließend durch die KI analysiert werden. Die Oberfläche stellt dafür eine einfache Eingabemaske mit einem Button zur Extraktion bereit (vgl. Abbildung 5-1).



Abbildung 5-1: Freitexteingabe und Start der KI-Verarbeitung

Nach der Verarbeitung werden die erkannten Informationen automatisch in strukturierte Felder übernommen. Der Nutzer kann diese Daten überprüfen und bei Bedarf manuell anpassen.

Falls Informationen fehlen oder nicht eindeutig erkannt werden, wird eine Rückfrage angezeigt. Der Nutzer kann die fehlenden Angaben direkt eingeben oder per Sprache ergänzen (vgl. Abbildung 5-2).



Abbildung 5-2: Rückfrage bei unklaren Angaben

Nach der Vervollständigung wird der fertige Kontaktbericht übersichtlich dargestellt. Die wichtigsten Informationen wie Ansprechpartner, Datum, Gesprächstyp und Inhalt werden strukturiert angezeigt. Anschließend kann der Bericht gespeichert werden. Die Eingabemaske unterstützt dabei die Bearbeitung aller Felder vor dem Speichern (vgl. Abbildung 5-3).

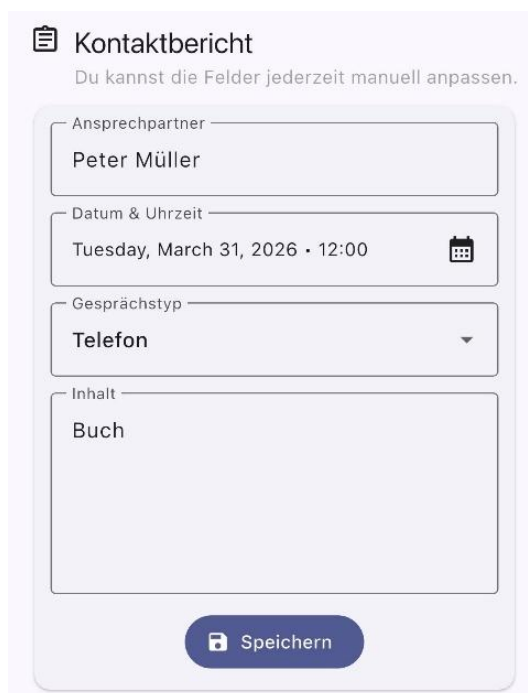


Abbildung 5-3: Anzeige eines fertigen Kontaktberichts

Gespeicherte Kontaktberichte werden in einer Liste dargestellt. Der Nutzer kann bestehende Berichte einsehen oder löschen sowie neue Berichte erstellen (vgl. Abbildung 5-4).

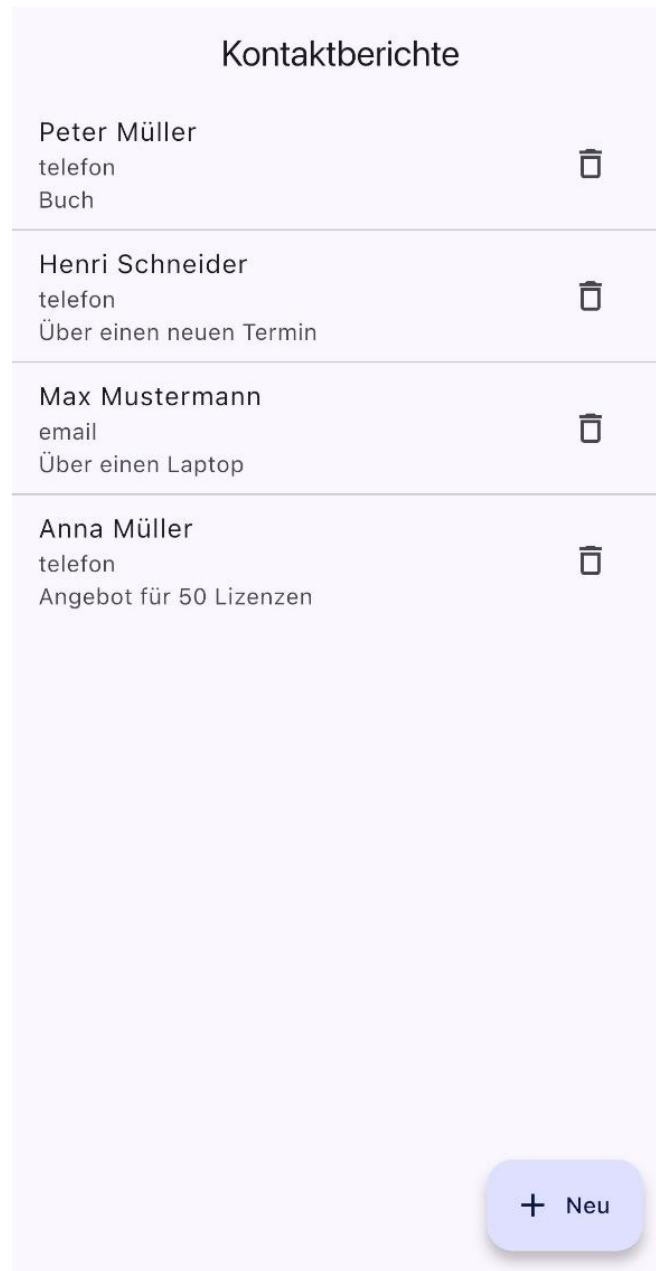


Abbildung 5-4: Übersicht gespeicherter Kontaktberichte

5.3 Umsetzung der Audioverarbeitung in AudioKit

Das Modul AudioKit realisiert die Verarbeitung von Spracheingaben sowie die Ausgabe von Sprache innerhalb der Anwendung. Es umfasst die Aufnahme von Audiodaten, deren Umwandlung in Text sowie die Ausgabe von Text als gesprochene Sprache.

Die Abbildung 5-5 zeigt Klassenstruktur sowie die Aufgaben der einzelnen Komponenten des Moduls.

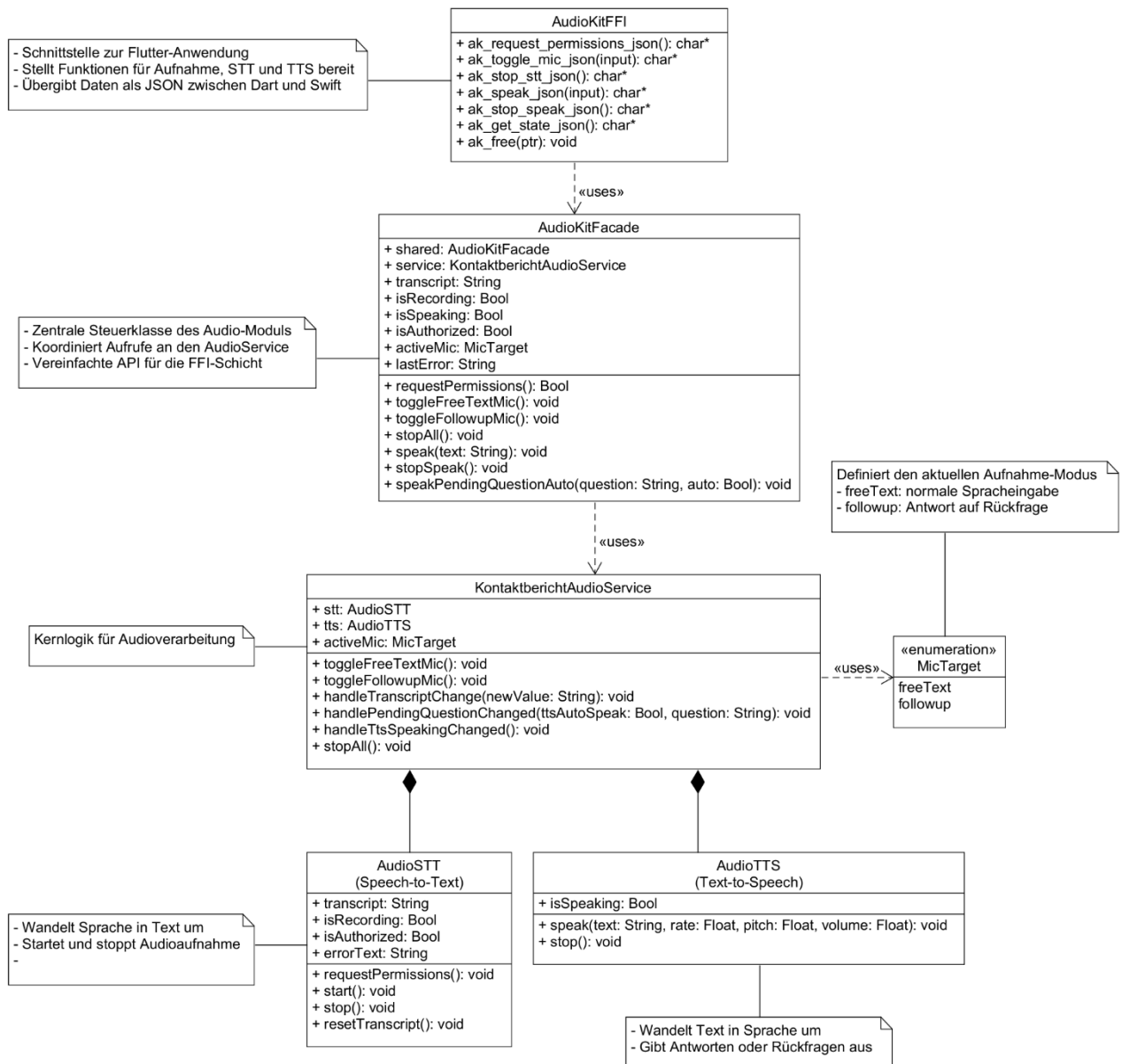


Abbildung 5-5: Klassendiagramm der Audioverarbeitung im AudioKit

Das Modul AudioKit realisiert die Verarbeitung von Spracheingaben sowie die Ausgabe von Sprache innerhalb der Anwendung. Es umfasst die Aufnahme von Audiodaten, deren Umwandlung in Text sowie die Ausgabe von Text als gesprochene Sprache.

Die Architektur des Moduls ist schichtartig aufgebaut. Über die FFI-Schnittstelle wird die Kommunikation mit der Flutter-Anwendung umgesetzt. Die zentrale Steuerung erfolgt über die

AudioKitFacade, welche die einzelnen Komponenten koordiniert und eine vereinfachte Schnittstelle bereitstellt.

Die eigentliche Verarbeitung wird im KontaktberichtAudioService umgesetzt. Dieser verbindet die STT- und TSS-Komponenten und steuert den Ablauf abhängig vom aktuellen Nutzungskontext, beispielsweise bei freier Spracheingabe oder bei der Beantwortung von Rückfragen.

Die Umwandlung von Sprache in Text erfolgt über die AudioSTT-Komponente unter Verwendung des iOS Speech Frameworks. Die Audioaufnahme wird dabei durch AVFoundation realisiert. Die Ausgabe von Sprache wird durch die AudioTTS-Komponente umgesetzt, die auf AVSpeechSynthesizer aus dem AVFoundation-Framework zurückgreift. Dadurch wird eine vollständig lokale Verarbeitung der Audiodaten ermöglicht.

Die Spracherkennung wurde in AudioSTT implementiert. In start() wird die Audioaufnahme mit AVAudioEngine gestartet und die Daten über SFSpeechAudioBufferRecognitionRequest an die Spracherkennung übergeben. Die Transkription erfolgt fortlaufend in recognitionTask(...) (vgl. Abbildung 5-6).

```
public func start() {
    guard !isRecording else { return }
    errorText = nil

    guard isAuthorized else {
        errorText = "Keine Berechtigung für Spracherkennung/Mikrofon."
        return
    }
    guard let recognizer, recognizer.isAvailable else {
        errorText = "Spracherkennung ist derzeit nicht verfügbar."
        return
    }

    request = SFSpeechAudioBufferRecognitionRequest()
    request?.shouldReportPartialResults = true

    let inputNode = audioEngine.inputNode
    let format = inputNode.outputFormat(forBus: 0)

    inputNode.removeTap(onBus: 0)
    inputNode.installTap(onBus: 0, bufferSize: 1024, format: format) { [weak self] buffer, _ in
        self?.request?.append(buffer)
    }

    do {
        audioEngine.prepare()
        try audioEngine.start()
    } catch {
        errorText = "AudioEngine konnte nicht starten: \(error.localizedDescription)"
        cleanupAudio()
        return
    }

    task = recognizer.recognitionTask(with: request!) { [weak self] result, error in
        guard let self else { return }
        if let result = result {
            self.transcript = result.bestTranscription.formattedString
        }
        if let error = error {
            self.errorText = "STT-Fehler: \(error.localizedDescription)"
            self.stop()
        }
    }

    isRecording = true
}
```

Abbildung 5-6: AudioSTT start()-Funktion

Die Methode `stop()` beendet die Aufnahme und gibt die Ressourcen wieder frei (vgl. Abbildung 5-7).

```
public func stop() {  
    guard isRecording else { return }  
    task?.cancel()  
    task = nil  
    request?.endAudio()  
    request = nil  
    cleanupAudio()  
    isRecording = false  
}
```

Abbildung 5-7: AudioSTT `stop()`-Funktion

Die Sprachausgabe wurde in AudioTTS mit AVSpeechSynthesizer umgesetzt und über die Fassade mit `speak()` und `stopSpeak()` gesteuert (vgl. Abbildung 5-8).

```
public func speak(_ text: String) {  
    service.tts.speak(text)  
    syncState()  
}  
  
public func stopSpeak() {  
    service.tts.stop()  
    syncState()  
}
```

Abbildung 5-8: Zentrale Fassade zur Steuerung der Audioverarbeitung (AudioKitFacade)

Die Anbindung an die Flutter-Anwendung erfolgt über AudioKitFFI mit JSON-basierten Funktionsaufrufen.

Die Abbildung 5-9 zeigt die Aktivitätsdiagramm der Audioverarbeitung im Modul AudioKit.

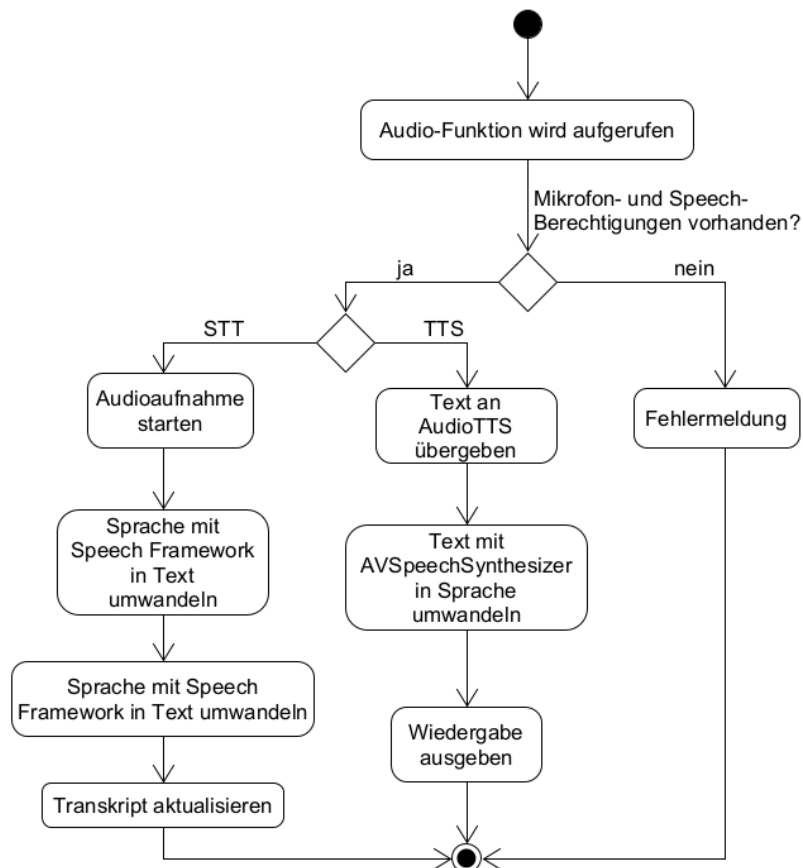


Abbildung 5-9: Aktivitätsdiagramm der Audioverarbeitung im Modul AudioKit

Das Aktivitätsdiagramm zeigt den Ablauf der Audioverarbeitung im Modul AudioKit. Nach dem Aufruf einer Audio-Funktion werden zunächst die erforderlichen Mikrofon- und Speech-Berechtigungen geprüft.

Anschließend wird abhängig von der gewählten Funktion zwischen Speech-to-Text (STT) und Text-to-Speech (TTS) unterschieden. Bei der Spracherkennung wird die Audioaufnahme gestartet und das Audiosignal mithilfe des Speech Frameworks in Text umgewandelt. Das erzeugte Transkript wird fortlaufend aktualisiert.

Bei der Sprachausgabe wird der eingegebene Text an die TTS-Komponente übergeben und mithilfe des AVSpeechSynthesizer aus dem AVFoundation-Framework in Sprache umgewandelt und ausgegeben.

Falls keine Berechtigungen vorliegen, wird eine entsprechende Fehlermeldung erzeugt und der Ablauf beendet.

5.4 Umsetzung der Kontaktbericht-Logik in KontaktberichtLogikKit

Das Modul KontaktberichtLogikKit bildet die zentrale Verarbeitungseinheit für die Erstellung von Kontaktberichten. Es übernimmt die Analyse von Freitexten, die strukturierte Aufbereitung der Informationen sowie die Speicherung der Ergebnisse.

Die Abbildung 5-10 zeigt die Klassenstruktur sowie die Aufgaben der einzelnen Komponenten des Moduls.

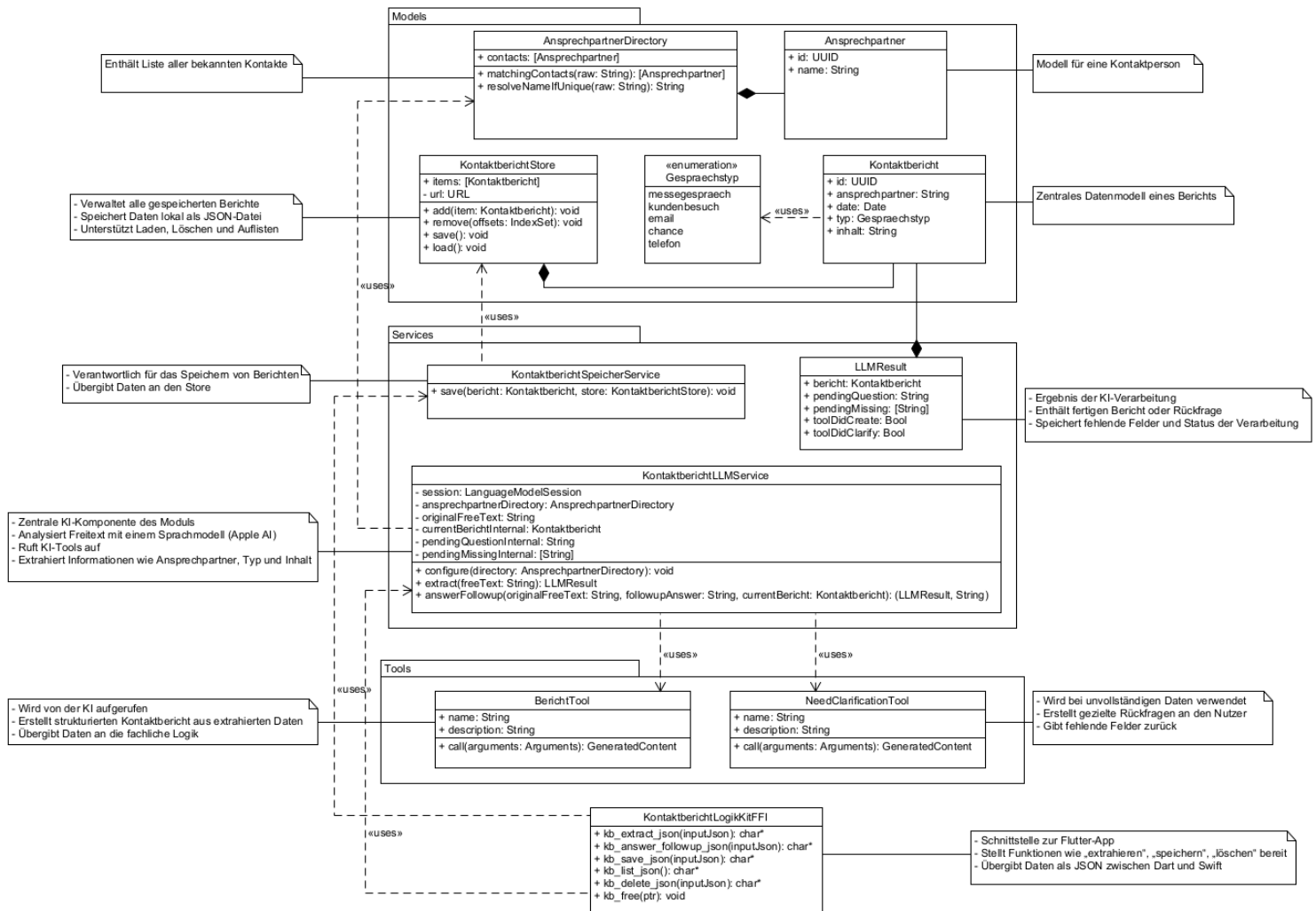


Abbildung 5-10: Klassendiagramm des Moduls KontaktberichtLogikKit

Im Mittelpunkt steht der KontaktberichtLLMService, der ein KI-Sprachmodell auf Basis von Apple Foundation Models nutzt. Die KI analysiert die Eingaben semantisch und ruft definierte Tools auf, um strukturierte Daten zu erzeugen oder fehlende Informationen zu erkennen. Dadurch wird die Verarbeitung nicht frei generiert, sondern durch klar definierte Abläufe gesteuert.

Die eigentliche Erstellung eines Berichts erfolgt über das BerichtTool, während das NeedClarificationTool bei unvollständigen oder mehrdeutigen Eingaben eingesetzt wird. Ergänzend dazu sorgen regelbasierte Komponenten, wie die Datumsverarbeitung und der Abgleich von Ansprechpartnern, für eine zusätzliche Validierung der Ergebnisse.

Die Speicherung der erzeugten Berichte erfolgt über den KontaktberichtSpeicherService in Kombination mit dem KontaktberichtStore, der die Daten lokal im Dateisystem verwaltet.

Die Kommunikation mit der Flutter-Anwendung wird über eine FFI-Schnittstelle realisiert. Dabei werden alle Eingaben und Ergebnisse in Form von JSON-Daten übertragen.

Obwohl Funktionsaufrufe über FFI grundsätzlich ohne Serialisierung erfolgen können, wird in dieser Arbeit bewusst eine JSON-basierte Datenübertragung verwendet. Dies ist notwendig, da komplexe Datenstrukturen wie Kontaktberichte nicht direkt über primitive FFI-Datentypen abgebildet werden können. Durch die Serialisierung in JSON wird ein einheitlicher und wartbarer Datenaustausch zwischen Dart und Swift ermöglicht.

Abbildung 5-11 zeigt exemplarisch die zentrale Einstiegsfunktion zur Verarbeitung von Freitext-Eingaben.

```
@cdecl("kb_extract_json")
public func kb_extract_json(_ inputJson: UnsafePointer<CChar>?) -> UnsafeMutablePointer<CChar>? {
    do {
        let input: ExtractInDTO = try decodeJSON(ExtractInDTO.self, inputJson)
        KBContext.shared.ensureConfigured()

        let r = runBlocking {
            try await KBContext.shared.llm.extract(from: input.freeText)
        }

        switch r {
        case .success(let res):
            let dto = LLMResultDTO(
                bericht: dtoFromModel(res.bericht),
                pendingQuestion: res.pendingQuestion,
                pendingMissing: res.pendingMissing,
                toolDidCreate: res.toolDidCreate,
                toolDidClarify: res.toolDidClarify
            )
            return encodeJSON(dto)
        case .failure(let e):
            return encodeErrorJSON(String(describing: e))
        }
    } catch {
        return encodeErrorJSON("decode_failed")
    }
}
```

Abbildung 5-11: Einstiegspunkt zur Analyse von Freitext-Eingaben (kb_extract_json)

Der dargestellte Code verdeutlicht den Ablauf der Verarbeitung. Zunächst wird die übergebene JSON-Struktur in ein internes Datenmodell überführt. Anschließend wird die Analyse durch den Sprachverarbeitungsdienst ausgeführt. Das Ergebnis wird in ein Data Transfer Object überführt und als JSON-String zurückgegeben.

Die Abbildung 5-12 zeigt die Aktivitätsdiagramm der Verarbeitung im Modul Kontaktbericht-LogikKit.

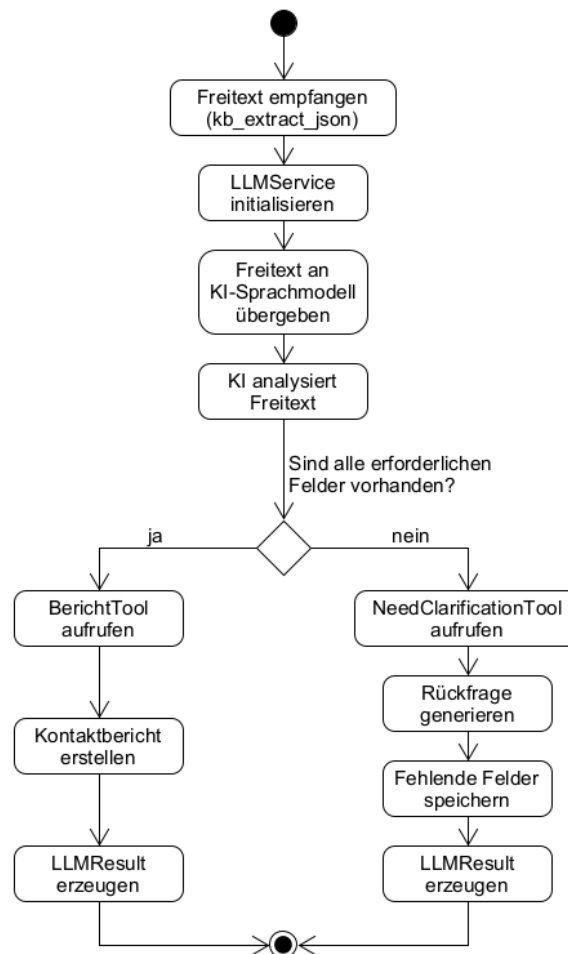


Abbildung 5-12: Aktivitätsdiagramm der Verarbeitung im Modul KontaktberichtLogikKit

Nach dem Empfang eines Freitextes wird dieser an den KontaktberichtLLMService übergeben und durch ein KI-Sprachmodell analysiert.

Abhängig vom Analyseergebnis wird entweder ein strukturierter Kontaktbericht erzeugt oder eine Rückfrage initiiert, wenn relevante Informationen fehlen oder nicht eindeutig erkannt werden. Die Erstellung eines Berichts erfolgt dabei über definierte Tools, wodurch die KI in einen kontrollierten Ablauf eingebunden wird.

Anschließend werden die Ergebnisse durch regelbasierte Komponenten weiterverarbeitet, beispielsweise zur Validierung von Datumsangaben und Ansprechpartnern sowie zur Bereinigung des Inhalts.

Das Ergebnis wird abschließend in Form eines strukturierten Objekts an die aufrufende Schicht zurückgegeben.

5.5 Implementierung der FFI-Integrationsschicht

Die Kommunikation zwischen der Flutter-Anwendung und den nativen Swift-Bibliotheken wurde mithilfe des Dart Foreign Function Interface (FFI) umgesetzt. Diese Integrationsschicht ermöglicht es, Funktionen der nativen Module direkt aus der Dart-Laufzeitumgebung aufzurufen und Ergebnisse strukturiert zurückzugeben.

Die Implementierung basiert auf einer klaren Trennung zwischen drei Ebenen: einer nativen C-kompatiblen Schnittstelle, automatisch generierten Dart-Bindings sowie einer Dart-seitigen Wrapper-Schicht zur komfortablen Nutzung der Funktionen.

Auf nativer Seite werden die Funktionen der Module als C-kompatible Methoden exportiert. Hierzu werden Funktionen mit dem Attribut `@cdecl` bereitgestellt, beispielsweise `kb_extract_json`, `kb_answer_followup_json` oder `kb_save_json`. Diese Funktionen bilden die Einstiegspunkte für die Verarbeitung von Freitexten, die Beantwortung von Rückfragen sowie die Persistenz von Kontaktberichten.

Auf Basis dieser Schnittstelle werden mithilfe des Tools `ffigen` automatisch Dart-Bindings erzeugt. Diese Bindings bilden die nativen Funktionen direkt als Dart-Funktionssignaturen ab und ermöglichen den Aufruf über die FFI-Schnittstelle.

Auf Dart-Seite wird zusätzlich eine Wrapper-Schicht implementiert, die die Interaktion mit den nativen Funktionen kapselt. Diese Schicht übernimmt insbesondere die Serialisierung und Deserialisierung von Daten sowie die Speicherverwaltung. So wird beispielsweise bei der Funktion `extractFromFreeText` der Eingabetext zunächst in ein JSON-Format überführt, an die native Funktion übergeben und das Ergebnis anschließend wieder in eine Dart-Datenstruktur zurückkonvertiert.

Die Datenübertragung zwischen Dart und Swift erfolgt vollständig über JSON-basierte Strukturen. Eingaben werden in Dart serialisiert und als Zeichenketten übergeben, während die nativen Module die Daten in interne Modelle umwandeln, verarbeiten und die Ergebnisse wiederum als JSON zurückgeben. Dieser Ansatz ermöglicht eine lose Kopplung zwischen den Laufzeitumgebungen und vereinfacht den Austausch komplexer Datenstrukturen.

Ein weiterer wichtiger Aspekt ist die explizite Speicherverwaltung. Da die nativen Funktionen Speicher für Rückgabewerte allokieren, wird dieser nach der Nutzung auf Dart-Seite über Funktionen wie `kb_free` wieder freigegeben, um Speicherlecks zu vermeiden.

Abbildung 5-13 zeigt exemplarisch die Dart-seitige Wrapper-Funktion zur Analyse von Freitext.

```
Map<String, dynamic> extractFromFreeText(String freeText) {  
    final inJson = jsonEncode({'freeText': freeText}).toNativeUtf8();  
    final outPtr = gen.kb_extract_json(inJson.cast());  
    malloc.free(inJson);  
  
    final outJson = _takeAndFree(outPtr);  
    return jsonDecode(outJson) as Map<String, dynamic>;  
}
```

Abbildung 5-13: Dart-Wrapper zur Nutzung der nativen FFI-Funktion zur Freitextanalyse

Der dargestellte Code verdeutlicht den vollständigen Ablauf der Integration. Zunächst werden Eingabedaten serialisiert und in einen nativen Speicherbereich übertragen. Anschließend erfolgt der Aufruf der nativen Funktion über die FFI-Schnittstelle. Das Ergebnis wird zurückgelesen, in eine Dart-Struktur überführt und der verwendete Speicher freigegeben.

Durch diese dreistufige Integrationsarchitektur wird eine klare Trennung zwischen nativer Implementierung und plattformübergreifender Anwendung erreicht. Gleichzeitig ermöglicht die Verwendung von FFI eine performante und direkte Nutzung systemnaher Funktionen innerhalb der Flutter-Anwendung.

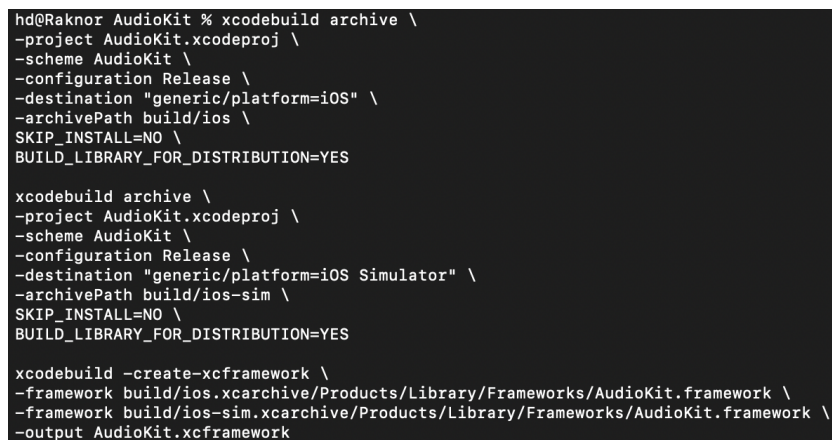
5.6 Erstellung der nativen XCFramework-Bibliotheken

Die Erstellung der Bibliotheken erfolgt in mehreren Schritten. Zunächst wird für jede Zielumgebung ein separates Framework archiviert. Hierzu werden jeweils Release-Builds für iOS-

Geräte und für den iOS-Simulator erzeugt. Dabei werden die Build-Optionen `SKIP_INSTALL=NO` und `BUILD_LIBRARY_FOR_DISTRIBUTION=YES` verwendet, um das Framework für die Weitergabe und spätere Integration bereitzustellen [22].

Für das Modul `AudioKit` erfolgt die Erstellung zunächst durch zwei getrennte Archivierungsvorgänge. Dabei wird jeweils eine Framework-Version für die Geräteumgebung und eine für die Simulatorumgebung erstellt. Anschließend werden beide Archive zu einem gemeinsamen `XCFramework` zusammengeführt.

Abbildung 5-14 zeigt exemplarisch die verwendeten Build-Kommandos zur Erstellung des Moduls `AudioKit` als `XCFramework`.



```
hd@Raknor AudioKit % xcodebuild archive \
-project AudioKit.xcodeproj \
-scheme AudioKit \
-configuration Release \
-destination "generic/platform=iOS" \
-archivePath build/ios \
SKIP_INSTALL=NO \
BUILD_LIBRARY_FOR_DISTRIBUTION=YES

xcodebuild archive \
-project AudioKit.xcodeproj \
-scheme AudioKit \
-configuration Release \
-destination "generic/platform=iOS Simulator" \
-archivePath build/ios-sim \
SKIP_INSTALL=NO \
BUILD_LIBRARY_FOR_DISTRIBUTION=YES

xcodebuild -create-xcframework \
-framework build/ios.xcarchive/Products/Library/Frameworks/AudioKit.framework \
-framework build/ios-sim.xcarchive/Products/Library/Frameworks/AudioKit.framework \
-output AudioKit.xcframework
```

Abbildung 5-14: Build-Kommandos zur Erstellung eines `XCFrameworks` für das Modul `AudioKit`

Der dargestellte Ablauf verdeutlicht, dass zunächst zwei separate Archive erzeugt werden. Im ersten Schritt wird das Framework für die Zielplattform `iOS` erstellt, im zweiten Schritt für den `iOS-Simulator`. Anschließend werden beide Framework-Versionen mit dem Befehl `xcodebuild -create-xcframework` zu einem gemeinsamen `XCFramework` zusammengeführt [22].

Das gleiche Vorgehen wurde auch für das Modul `KontaktberichtLogikKit` angewendet. Dadurch stehen beide nativen Bibliotheken in einem Format zur Verfügung, das sowohl von physischen Geräten als auch von Simulatorumgebungen genutzt werden kann.

Die Bereitstellung als `XCFramework` ist eine wichtige Voraussetzung für die in Abschnitt 5.5 beschriebene FFI-Integration. Erst durch diese Paketierung können die nativen Bibliotheken durch die Flutter-seitige Build- und Bindingschicht eingebunden und zur Laufzeit geladen werden. Im Fall des Moduls `KontaktberichtLogikKit` wird dies zusätzlich durch die build-seitige

Auswahl des passenden XCFramework-Slices unterstützt, sodass je nach Zielarchitektur automatisch die geeignete Framework-Variante verwendet wird.

Durch dieses Vorgehen wird eine saubere Trennung zwischen nativer Implementierung, Paketierung und plattformübergreifender Integration erreicht. Gleichzeitig erhöht die Nutzung von XCFrameworks die Wartbarkeit und Wiederverwendbarkeit der nativen Module innerhalb des Gesamtsystems.

5.7 Zusammenspiel der Systemkomponenten

Der Verarbeitungsablauf beginnt mit der Eingabe von Gesprächsinhalten über die Flutter-basierte Benutzeroberfläche. Bei Spracheingaben werden die Audiodaten durch das Modul AudioKit aufgenommen und in Text transkribiert.

Der erzeugte Freitext wird über die FFI-Schnittstelle an das Modul KontaktberichtLogikKit übergeben, das die Analyse durchführt und strukturierte Informationen extrahiert. Die Ergebnisse werden anschließend an die Flutter-Anwendung zurückgegeben und in der Benutzeroberfläche dargestellt.

Unvollständige Eingaben können durch Rückfragen ergänzt werden, wodurch ein iterativer Verarbeitungsprozess entsteht. Abschließend kann der Kontaktbericht lokal gespeichert werden.

5.8 Zusammenfassung der prototypischen Umsetzung

Die prototypische Umsetzung zeigt, dass eine modulare Architektur aus Flutter-basierter Benutzeroberfläche und nativen Swift-Komponenten erfolgreich realisiert werden kann.

Durch die FFI-basierte Integration lassen sich systemnahe Funktionen effizient einbinden, während die Bereitstellung als XCFramework eine flexible Nutzung auf verschiedenen Zielarchitekturen ermöglicht.

Der entwickelte Prototyp erfüllt damit die grundlegenden Anforderungen an eine offlinefähige, KI-gestützte Unterstützung zur Erstellung von Kontaktberichten.

6 Evaluation

In diesem Kapitel wird der entwickelte Prototyp eines KI-basierten Kontaktbericht-Assistenten evaluiert. Ziel der Evaluation ist es zu untersuchen, inwieweit die in Kapitel 3 definierten funktionalen und nicht-funktionalen Anforderungen durch die prototypische Umsetzung erfüllt werden. Die Bewertung erfolgt auf Grundlage der implementierten Systemfunktionen sowie der praktischen Nutzung des Prototyps während der Entwicklung.

6.1 Evaluationsmethodik

Die Evaluation des entwickelten Systems erfolgt qualitativ anhand der in Kapitel 3 definierten Anforderungen. Dabei wird untersucht, inwieweit die implementierten Funktionen des Prototyps die vorgesehenen Anforderungen erfüllen und wie gut sich die gewählte Architektur für die Umsetzung der Anwendung eignet.

Da es sich um eine prototypische Implementierung handelt, liegt der Fokus auf der funktionalen Demonstration zentraler Systemkomponenten. Dazu gehören insbesondere die Erfassung von Spracheingaben, die automatische Analyse von Gesprächsinhalten sowie die Erstellung und Speicherung strukturierter Kontaktberichte.

Zur Bewertung der Spracheingabe wurden unterschiedliche Eingabeszenarien betrachtet, um die Robustheit der Spracherkennung und Weiterverarbeitung zu untersuchen. Dazu zählen unter anderem klar formulierte Standardsätze, unvollständige oder grammatikalisch fehlerhafte Äußerungen sowie Eingaben mit Füllwörtern wie „äh“. Zusätzlich wurden verschiedene Sprechweisen berücksichtigt, beispielsweise unterschiedliche Sprechgeschwindigkeiten sowie sprachliche Variationen wie Akzente oder nicht-muttersprachliche Ausdrucksweisen.

Darüber hinaus wurden Szenarien mit unterschiedlich strukturierten Texteingaben analysiert, um die Leistungsfähigkeit der KI-gestützten Informationsverarbeitung zu bewerten. Hierzu gehören sowohl klar strukturierte Aussagen als auch unstrukturierte, mehrdeutige oder teilweise unvollständige Eingaben.

Die Bewertung erfolgt auf Basis der beobachteten Systemreaktionen, insbesondere hinsichtlich der Korrektheit der Transkription, der Qualität der Informationsextraktion sowie der Stabilität der Anwendung während der Verarbeitung.

6.2 Bewertung der funktionalen Anforderungen

Die in Kapitel 3 definierten funktionalen Anforderungen wurden im Rahmen, der in Abschnitt 6.1 beschriebenen Testszenarien systematisch überprüft. Dabei wurde analysiert, inwieweit die implementierten Funktionen des Prototyps die vorgesehenen Anforderungen erfüllen und wie zuverlässig die Verarbeitung unter unterschiedlichen Eingabebedingungen funktioniert.

Die Erfassung von Gesprächsinhalten (F1) wurde sowohl über manuelle Texteingabe als auch über Spracheingabe realisiert. In den Tests zeigte sich, dass die Spracheingabe bei klarer Aussprache und strukturierter Formulierung zuverlässig funktioniert. Gesprochene Inhalte werden in diesen Fällen korrekt in Text umgewandelt und können direkt weiterverarbeitet werden. Auch die manuelle Texteingabe ermöglicht eine stabile und fehlerfreie Erfassung von Gesprächsinhalten. Die Anforderung wird daher insgesamt als erfüllt bewertet.

Die KI-gestützte Strukturierung von Freitext (F3) zeigt differenzierte Ergebnisse. Bei klar formulierten und inhaltlich strukturierten Eingaben gelingt die Extraktion relevanter Informationen wie Datum, Ansprechpartner und Gesprächsinhalte zuverlässig. Die Informationen werden korrekt in das zugrunde liegende Datenmodell überführt und dem Benutzer strukturiert angezeigt. Einschränkungen zeigen sich jedoch bei unvollständigen, mehrdeutigen oder sprachlich unstrukturierten Eingaben. Insbesondere bei der Verwendung von Füllwörtern, unklaren Satzstrukturen oder fehlendem Kontext kommt es teilweise zu unvollständigen oder fehlerhaften Extraktionen. In diesen Fällen ist eine manuelle Nachbearbeitung erforderlich. Die Anforderung wird daher als teilweise erfüllt bewertet.

Die strukturierte Darstellung der extrahierten Informationen (F4) wird durch das definierte Datenmodell sowie die Benutzeroberfläche zuverlässig umgesetzt. Die erkannten Inhalte werden übersichtlich in Formularfeldern dargestellt und können vom Benutzer nachvollzogen werden. Diese Anforderung wird als erfüllt bewertet.

Die Nachbearbeitung durch den Nutzer (F5) ist vollständig gegeben. Die automatisch erkannten Inhalte können im Interface überprüft, korrigiert und ergänzt werden. Dies ermöglicht eine flexible Anpassung der erzeugten Kontaktberichte und trägt zur Verbesserung der Datenqualität bei. Die Anforderung wird daher als erfüllt bewertet.

Die lokale Speicherung von Kontaktberichten (F6) funktioniert stabil und zuverlässig. Erstellte Berichte können gespeichert, geladen und erneut bearbeitet werden. Die Datenhaltung erfolgt

vollständig lokal auf dem Gerät, wodurch auch die Anforderungen an Offline-Fähigkeit und Datenschutz unterstützt werden. Die Anforderung wird als erfüllt bewertet.

Zur zusammenfassenden Darstellung der Bewertung der funktionalen Anforderungen zeigt Tabelle 6-1 eine Übersicht der einzelnen Anforderungen, deren Umsetzung im Prototyp sowie die resultierende Bewertung.

ID	Anforderung	Umsetzung im Prototyp	Bewertung
F1/F2	Erfassung von Gesprächsinhalten	Spracheingabe über integrierte STT-Funktion sowie manuelle Texteingabe	Erfüllt
F3	KI-gestützte Strukturierung von Freitext	Analyse der eingegebenen Texte zur Extraktion relevanter Informationen für Kontaktberichte	Teilweise erfüllt (strukturierte Inhalte zuverlässig erkannt, bei unklaren oder unvollständigen Eingaben eingeschränkt)
F4	Strukturierter Bericht	Darstellung der Informationen im Interface	Erfüllt
F3	Offline-Verarbeitung	Sprachverarbeitung und Analyse erfolgen lokal auf dem Gerät ohne Cloud-Dienste	Erfüllt
F5	Nachbearbeitung durch den Nutzer	Erkannte Inhalte können im Interface überprüft und manuell angepasst werden	Erfüllt
F6	Lokale Speicherung von Berichten	Kontaktberichte werden lokal gespeichert und können erneut geladen oder bearbeitet werden	Erfüllt

Tabelle 6-1: Bewertung der funktionalen Anforderungen

Die Tabelle verdeutlicht, dass die meisten funktionalen Anforderungen vollständig erfüllt werden konnten. Lediglich die KI-gestützte Analyse von Freitext weist Einschränkungen auf, insbesondere bei unstrukturierten oder mehrdeutigen Eingaben. Insgesamt zeigt sich jedoch, dass

der entwickelte Prototyp die zentralen Funktionalitäten eines KI-basierten Kontaktbericht-Assistenten erfolgreich umsetzt.

6.3 Bewertung der nicht-funktionalen Anforderungen

Neben den funktionalen Anforderungen wurden auch mehrere nicht-funktionale Anforderungen im Rahmen der in Abschnitt 6.1 beschriebenen Testszenarien bewertet. Dabei wurde insbesondere untersucht, wie sich das System hinsichtlich Performance, Benutzerfreundlichkeit, Wartbarkeit und Datenschutz im praktischen Einsatz verhält.

Ein wesentlicher Aspekt ist die Offline-Fähigkeit der Anwendung (NF1). Alle Verarbeitungsprozesse, einschließlich der Sprachtranskription und der Analyse von Gesprächsinhalten, erfolgen lokal auf dem mobilen Endgerät. In den Tests konnte gezeigt werden, dass die Anwendung vollständig ohne Internetverbindung funktionsfähig ist. Diese Anforderung wird daher als erfüllt bewertet.

Auch die Datenschutzanforderung (NF2) wird erfüllt, da keine Daten an externe Server übertragen werden. Sämtliche Sprach- und Textdaten werden ausschließlich lokal verarbeitet und gespeichert.

Die Performance (NF3) wurde anhand der Reaktionszeiten während der Nutzung bewertet. In den Tests zeigte sich, dass die Verarbeitung von Spracheingaben sowie die Analyse von Texten in der Regel innerhalb weniger Sekunden erfolgt. Zudem reagiert die Benutzeroberfläche ohne wahrnehmbare Verzögerung auf Benutzerinteraktionen. Die Anforderung wird daher als erfüllt bewertet.

Die Benutzerfreundlichkeit (NF4) wurde anhand der Interaktion mit der Benutzeroberfläche sowie der notwendigen Schritte zur Erstellung eines Kontaktberichts bewertet. In den Tests zeigte sich, dass grundlegende Funktionen wie Spracheingabe, Anzeige der Ergebnisse und Nachbearbeitung intuitiv nutzbar sind. Einschränkungen bestehen jedoch in Situationen, in denen automatisch extrahierte Inhalte unvollständig oder fehlerhaft sind und eine manuelle Korrektur erforderlich wird. Zudem kann die Interaktion bei unklaren Spracheingaben zusätzliche Rückfragen erfordern. Aus diesen Gründen wird die Benutzerfreundlichkeit insgesamt als teilweise erfüllt bewertet.

Die Wartbarkeit und Erweiterbarkeit (NF5) wird durch die modulare Architektur des Systems unterstützt. Die klare Trennung zwischen Flutter-Oberfläche, AudioKit und KontaktberichtLogikKit ermöglicht eine unabhängige Weiterentwicklung einzelner Komponenten. Diese Anforderung wird als erfüllt bewertet.

Die Plattformintegration (NF6) wird durch die Nutzung nativer iOS-Frameworks wie Speech und AVFoundation realisiert. In den Tests konnte gezeigt werden, dass der Zugriff auf Mikrofon- und Sprachfunktionen zuverlässig funktioniert. Die Anforderung wird daher als erfüllt bewertet.

Zur zusammenfassenden Darstellung zeigt Tabelle 6-2 die Bewertung der nicht-funktionalen Anforderungen.

ID	Anforderung	Umsetzung im System	Bewertung
NF1	Offline-Fähigkeit	Verarbeitung erfolgt vollständig lokal ohne Internet	Erfüllt
NF2	Datenschutz	Verarbeitung aller Daten lokal auf dem Endgerät	Erfüllt
NF3	Performance	Nutzung nativer Swift-Module für Audio- und Textverarbeitung	Erfüllt
NF4	Benutzerfreundlichkeit	Flutter-basierte Oberfläche mit Eingabe- und Bearbeitungsfunktionen	Teilweise erfüllt (manuelle Korrekturen erforderlich)
NF5	Wartbarkeit und Erweiterbarkeit	Modulare Architektur mit getrennten Komponenten (Flutter, AudioKit, KontaktberichtLogikKit)	Erfüllt
NF6	Plattformintegration	Zugriff auf iOS-Audio- und Sprachschnittstellen über native Module	Erfüllt

Tabelle 6-2: Bewertung der nicht-funktionalen Anforderungen

Die Tabelle verdeutlicht, dass die meisten nicht-funktionalen Anforderungen erfüllt werden konnten. Einschränkungen bestehen insbesondere im Bereich der Benutzerfreundlichkeit, die

stark von der Qualität der Eingaben und der Genauigkeit der automatischen Verarbeitung abhängt.

6.4 Grenzen der Evaluation

Da es sich bei der entwickelten Anwendung um einen prototypischen Demonstrator handelt, sind bestimmte Einschränkungen der Evaluation zu berücksichtigen.

Die Bewertung basiert primär auf einer qualitativen Analyse der implementierten Funktionen. Eine umfassende empirische Untersuchung mit einer größeren Anzahl von Nutzern wurde im Rahmen dieser Arbeit nicht durchgeführt.

Darüber hinaus ist die aktuelle Implementierung auf die Plattform iOS beschränkt, da die nativen Komponenten in Swift implementiert wurden. Für eine vollständige plattformübergreifende Nutzung wären zusätzliche Implementierungen für weitere Plattformen erforderlich.

Trotz dieser Einschränkungen zeigt der entwickelte Prototyp, dass die grundlegenden Anforderungen eines KI-basierten Kontaktbericht-Assistenten umgesetzt werden können und eine geeignete Grundlage für weiterführende Entwicklungen besteht.

6.5 Bewertung der Systemarchitektur

Neben der funktionalen Bewertung des Prototyps wird auch die zugrunde liegende Systemarchitektur betrachtet. Ziel dieser Bewertung ist es zu untersuchen, inwieweit die gewählte Architektur die in Kapitel 3 definierten funktionalen und nicht-funktionalen Anforderungen unterstützt.

Die entwickelte Architektur basiert auf einer klaren Trennung zwischen Benutzeroberfläche und nativer Verarbeitungslogik. Während die Benutzeroberfläche mithilfe des Flutter-Frameworks umgesetzt wurde, erfolgt die Verarbeitung von Audio- und Textdaten in nativen Swift-Modulen. Diese Trennung ermöglicht eine modulare Struktur des Systems und erleichtert die Erweiterung einzelner Komponenten.

Ein weiterer Vorteil der Architektur liegt in der Verwendung des Dart Foreign Function Interface (FFI) zur Integration nativer Bibliotheken. Dadurch können Funktionen aus den nativen Swift-Modulen direkt aus der Flutter-Anwendung aufgerufen werden, ohne den Umweg über

nachrichtenbasierte Plattformkanäle. Dies reduziert den Kommunikations-Overhead und ermöglicht eine performante Verarbeitung von Audio- und Textdaten.

Zur zusammenfassenden Bewertung der Anforderungserfüllung zeigt Tabelle 6-4 die Zuordnung der in Kapitel 3 definierten Anforderungen zu den jeweiligen Architekturkomponenten sowie deren konkreter Umsetzung im System. Dadurch wird verdeutlicht, wie die einzelnen Anforderungen durch das Zusammenspiel der Systemkomponenten realisiert werden.

ID	Anforderung	Architekturkomponente	Implementierung
F1	Spracheingabe	AudioKit	Audioaufnahme und STT
F2	Texteingabe	Flutter UI	Formularbasierte Texteingabe
F3	KI-Analyse von Freitext	KontaktberichtLogikKit	LLMService und Texteingabe
F4	Strukturierte Berichte	KontaktberichtLogikKit + Flutter UI	Datenmodell Kontaktbericht
F5	Nachbearbeitung	Flutter UI	Formularfelder zur Bearbeitung
F6	Lokale Speicherung	KontaktberichtLogikKit	KontaktberichtSpeicherService
NF1	Offline-Verarbeitung	Native Module	On-device Verarbeitung
NF2	Datenschutz	Architekturdesign	Keine Cloud-Kommunikation
NF3	Performance	Dart FFI + Swift	Native Verarbeitung
NF4	Benutzerfreundlichkeit	Flutter UI	Intuitive Oberfläche
NF5	Wartbarkeit	Modularisierung	AudioKit + KontaktberichtLogikKit
NF6	Plattformintegration	iOS-Frameworks	Speech Framework, AVFoundation

Tabelle 6-3: Zuordnung von Anforderungen, Architekturkomponenten und Implementierung

Die Tabelle verdeutlicht, dass die funktionalen Anforderungen durch eine klare Aufteilung auf spezialisierte Module umgesetzt werden. So wird beispielsweise die Spracheingabe (F1) durch das Modul AudioKit realisiert, während die KI-gestützte Analyse von Freitexten (F3) im Modul KontaktberichtLogikKit implementiert ist. Die Benutzerinteraktion sowie die Nachbearbeitung (F2, F5) werden hingegen durch die Flutter-basierte Benutzeroberfläche bereitgestellt.

Auch die nicht-funktionalen Anforderungen werden durch die Architektur gezielt unterstützt. Die Offline-Fähigkeit (NF1) sowie der Datenschutz (NF2) werden durch die vollständige lokale Verarbeitung der Daten gewährleistet. Die Performance (NF3) profitiert insbesondere von der Nutzung nativer Swift-Komponenten in Kombination mit der FFI-Schnittstelle. Die Wartbarkeit (NF5) wird durch die modulare Struktur des Systems unterstützt, während die Plattformintegration (NF6) durch die Nutzung systemnaher iOS-Frameworks realisiert wird.

Die folgende Tabelle fasst die Bewertung zentraler Architekturmerkmale zusammen.

Architekturmerkmal	Bewertung (1=niedrig, 5=hoch)	Begründung
Modularität	5	klare Trennung von Flutter-UI, AudioKit und KontaktberichtLogikKit
Wartbarkeit	4	Komponenten sind getrennt, Schnittstellen klar definiert
Performance	5	native Verarbeitung über Swift und FFI
Erweiterbarkeit	3	Erweiterungen möglich, aber Plattformabhängigkeit
Plattformunabhängigkeit	2	derzeit nur iOS implementiert

Tabelle 6-4: Bewertung der Architekturmerkmale

Die Tabelle 6.3 zeigt die qualitative Bewertung zentraler Architekturmerkmale des entwickelten Systems. Dabei wird deutlich, dass insbesondere Modularität und Performance der Architektur positiv bewertet werden können. Gleichzeitig zeigt sich, dass die Plattformunabhängigkeit derzeit eingeschränkt ist, da die Implementierung der nativen Komponenten auf iOS basiert.

Zur besseren Veranschaulichung dieser Bewertung werden die Ergebnisse zusätzlich grafisch dargestellt.

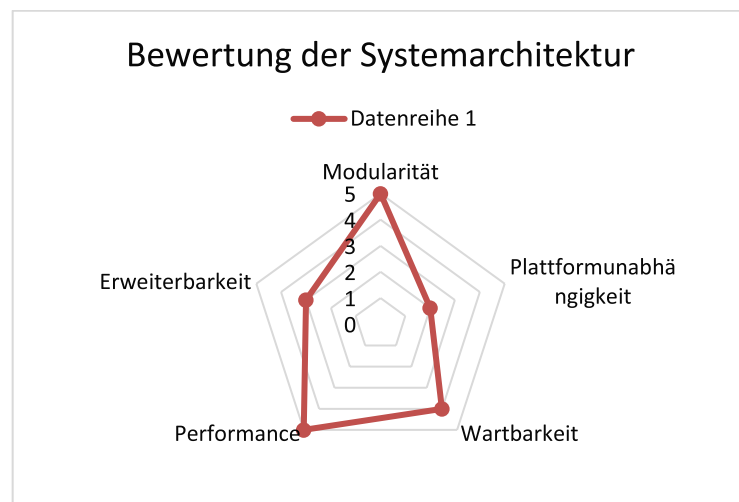


Abbildung 6-1: Qualitative Bewertung der Architekturmerkmale des Kontaktbericht-Assistenten

Die Abbildung 6.1 zeigt eine grafische Darstellung der qualitativen Bewertung zentraler Architekturmerkmale des entwickelten Systems. Besonders deutlich werden dabei die hohe Modularität und Performance der Architektur, die durch die Trennung zwischen Flutter-basierter Benutzeroberfläche und nativen Swift-Modulen erreicht wird. Einschränkungen bestehen vor allem hinsichtlich der Plattformunabhängigkeit, da die aktuelle Implementierung auf iOS-Geräte beschränkt ist.

6.6 Probleme bei der Implementierung

Während der Implementierung traten einige Herausforderungen auf. Ein zentrales Problem war die Formulierung der Prompts für das KI-Modell. Wenn ein Prompt nicht klar oder nicht genau genug ist, kann die KI die Ergebnisse erzeugen, die nicht dem gewünschten Verhalten entsprechen.

chen. Dadurch können Informationen falsch interpretiert, unvollständig extrahiert oder unerwünscht ergänzt werden. Aus diesem Grund mussten die Prompts mehrfach angepasst und getestet werden.

Ein weiteres Problem betrifft den Zugriff auf das Mikrofon. Für die Sprachaufnahme ist eine Benutzererlaubnis erforderlich. Wird diese nicht erteilt, kann die Audioaufnahme nicht gestartet werden und die Spracherkennung funktioniert nicht. Daher musste eine entsprechende Berechtigungsprüfung implementiert und mögliche Fehlerfälle berücksichtigt werden.

6.7 Zusammenfassung der Evaluation

Die Evaluation des entwickelten Prototyps zeigt, dass die in Kapitel 3 definierten zentralen Anforderungen grundsätzlich umgesetzt werden konnten. Insbesondere die Erfassung von Gesprächsinhalten über Spracheingabe sowie die automatische Analyse und Strukturierung der Inhalte konnten prototypisch realisiert werden. Auch die lokale Verarbeitung der Daten erfüllt die Anforderungen an Datenschutz und Offline-Fähigkeit.

Darüber hinaus bestätigt die Analyse der Systemarchitektur, dass die gewählte Kombination aus einer Flutter-basierten Benutzeroberfläche und nativen Swift-Modulen eine geeignete Grundlage für die Umsetzung eines solchen Systems darstellt. Die modulare Struktur ermöglicht eine klare Trennung zwischen Präsentationslogik und Verarbeitungslogik und unterstützt eine gute Wartbarkeit sowie Erweiterbarkeit der Anwendung.

Gleichzeitig zeigt die Evaluation auch bestehende Einschränkungen des Prototyps. Dazu zählen insbesondere die derzeitige Plattformbindung an iOS sowie mögliche Verbesserungen hinsichtlich der Benutzeroberfläche und der Analysegenauigkeit der Textverarbeitung. Zudem traten während der Implementierung Herausforderungen bei der Formulierung von KI-Prompts sowie beim Umgang mit Mikrofonberechtigungen auf, die die Funktionalität der Sprachverarbeitung beeinflussen können. Insbesondere zeigte sich, dass die verwendeten On-Device-Sprachmodelle hinsichtlich ihrer Analysegenauigkeit nicht die Qualität cloudbasierter Modelle erreichen. Dies führt dazu, dass die erzeugten Ergebnisse teilweise unvollständig oder ungenau sind. Eine Verbesserung der Ergebnisse erfordert daher eine gezielte Optimierung der Prompts sowie eine weitere Anpassung der Verarbeitungsschritte.

Insgesamt zeigt die Evaluation jedoch, dass der entwickelte Prototyp eine funktionierende Grundlage für einen KI-basierten Kontaktbericht-Assistenten darstellt und das zugrunde liegende Architekturkonzept für eine weitere Weiterentwicklung geeignet ist. Im folgenden Kapitel werden die Ergebnisse der Arbeit zusammengefasst und mögliche zukünftige Erweiterungen diskutiert.

7 Zusammenfassung, Fazit und Ausblick

In diesem Kapitel werden die wichtigsten Ergebnisse der Arbeit zusammengefasst und bewertet. Außerdem werden die Grenzen des Prototyps beschrieben und mögliche Weiterentwicklungen vorgestellt.

7.1 Zusammenfassung

Ziel dieser Arbeit ist die Entwicklung eines offlinefähigen, KI-basierten Assistenten zur Erstellung von Kontaktberichten. Dafür wird eine Systemarchitektur genutzt, die eine plattformübergreifende Benutzeroberfläche mit nativen Modulen verbindet.

Der Prototyp ermöglicht die Erfassung von Gesprächsinhalten durch Sprache und Text sowie deren automatische Analyse und strukturierte Darstellung. Die Verarbeitung erfolgt vollständig lokal auf dem mobilen Gerät.

Die Ergebnisse zeigen, dass die meisten Anforderungen erfüllt werden. Besonders die Offline-Fähigkeit, der Datenschutz und die Integration nativer Komponenten funktionieren zuverlässig. Einschränkungen bestehen vor allem bei der automatischen Auswertung von unklaren oder unvollständigen Eingaben.

7.2 Fazit

Die entwickelte Lösung zeigt, dass die lokale Verarbeitung von Sprachdaten technisch möglich ist. Dabei können sowohl Datenschutzerfordernungen als auch Anforderungen an die Systemintegration erfüllt werden.

Ein wichtiger Vorteil ist die modulare Architektur. Sie ermöglicht eine klare Trennung zwischen Benutzeroberfläche, Audioverarbeitung und Verarbeitungslogik. Die Nutzung von Dart FFI bietet eine gute Möglichkeit, native Funktionen effizient einzubinden.

Gleichzeitig hat der Prototyp auch Grenzen. Die Qualität der automatischen Auswertung hängt von der Formulierung der Eingaben ab. Bei unklaren oder unvollständigen Aussagen sind zusätzliche Rückfragen nötig. Außerdem ist die Integration und Pflege nativer Komponenten mit höherem Aufwand verbunden.

7.3 Ausblick

Für zukünftige Arbeiten gibt es mehrere Möglichkeiten zur Weiterentwicklung. Eine wichtige Erweiterung ist die Verbesserung der Sprachverarbeitung, zum Beispiel durch genauere Modelle und eine bessere Verarbeitung unvollständiger Eingaben.

Auch die Benutzerinteraktion kann weiter verbessert werden, etwa durch natürlichere Dialoge oder eine bessere Unterstützung bei der Eingabe. Zusätzlich könnten Funktionen wie die Anbindung an CRM-Systeme oder die Synchronisation von Daten integriert werden.

Ein weiterer Ansatz ist die Optimierung der Systemarchitektur, zum Beispiel durch bessere Performance oder einfachere Wartung.

Insgesamt bietet der entwickelte Prototyp eine gute Grundlage für zukünftige Entwicklungen im Bereich offlinefähiger, KI-basierter Assistenzsysteme.

8 Literaturverzeichnis

- [1] Apple Inc., The Swift Programming Language, Cupertino, CA: Apple Inc., 2023.
- [2] K. Lee, Beginning Swift Programming, Birmingham, UK: Packt Publishing, 2021.
- [3] J. Hoffman, Mastering Swift, Birmingham, UK: Packt Publishing, 2019.
- [4] D. M. R. Brian W. Kernighan, The C Programming Language. 2nd Edition, Englewood Cliff, New Jersey 07632: Prentice Hall, 1988.
- [5] J. Bloch, Effective Java, 3rd Edition, Boston: Addison-Wesley Professional, 2018.
- [6] A. Van der Lee, „Advanced Swift,“ [Online]. Available: <https://www.avanderlee.com/swift/advanced-swift/>. [Zugriff am März 2026].
- [7] F. Cheng, Flutter for Beginners, Birmingham, UK: Packt Publishing, 2019.
- [8] D. Team, „C interop using dart:ffi,“ 2024. [Online]. Available: <https://dart.dev/interop/c-interop>. [Zugriff am Januar 2026].
- [9] Oracle, „Java Exception Handling,“ [Online]. Available: <https://docs.oracle.com/javase/tutorial/essential/exceptions/>. [Zugriff am 3 2026].
- [10] B. Stroustrup, The C++ Programming Language (4th Edition), Boston: Addison-Wesley Professional, 2013.
- [11] Google, „Flutter Dokumentation,“ 2024. [Online]. Available: <https://docs.flutter.dev/>. [Zugriff am Januar 2026].
- [12] R. Payne, Beginning App Development with Flutter, Birmingham, UK: Packt Publishing , 2019.
- [13] S. H. a. T. A. M. Henning Heitkötter, „Evaluating Cross-Platform Development Approaches for Mobile Applications,“ University of Münster, Münster, Germany, 2013.

- [14] Flutter Clutter, „StatelessWidget vs. StatefulWidget,“ 8 8 2020. [Online]. Available: <https://www.flutterclutter.dev/flutter/basics/statelesswidget-vs-statefulwidget/2020/1195/>. [Zugriff am 3 2026].
- [15] Flutter, „Impeller rendering engine,“ 24 3 2026. [Online]. Available: <https://docs.flutter.dev/perf/impeller>. [Zugriff am 4 2026].
- [16] M. Hajian, Flutter Engineering: Become a Flutter Professional Using Tactics Explained from a Software Engineer's Perspective, Staten House, 2024.
- [17] D. J. & J. H. Martin, Speech and Language Processing, Standford, CA: Stanford University, 2023.
- [18] S. R. & P. Norvig, „Artificial Intelligence: A Modern Approach,“ Pearson Education, Inc., Hoboken, NJ, 2021.
- [19] Sommerville, Software Engineering, 10th Edition, Boston, MA: Pearson, 2016.
- [20] P. C. R. K. Len Bass, Software Architecture in Practice, 3rd Edition, Boston, MA: Addison-Wesley, 2021.
- [21] M. Kleppmann, Designing Data-Intensive Applications, Sebastopol, CA: O'Reilly Media, 2017.
- [22] E. M. Amira, „Build your first XCFramework -How to create an iOS framework Pt 1,“ 3 5 2022. [Online]. Available: <https://medium.com/@elmoezamira/build-your-first-xcframework-how-to-create-an-ios-framework-pt-1-d1a889fdb4od>. [Zugriff am 2 2026].
- [23] Flutter, „Build for and integrate with multiple platforms,“ 8 12 2025. [Online]. Available: <https://docs.flutter.dev/platform-integration>. [Zugriff am 2 2026].
- [24] Google Developers, „Offline-orientierte App erstellen,“ 27 7 2025. [Online]. Available: <https://developer.android.com/topic/architecture/data-layer/offline-first?hl=de>. [Zugriff am 3 2026].

- [25] K. H. Frank Buschmann, Pattern-Oriented Software Architecture, Volumn 1, Chichester, UK: Wiley, 1996.
- [26] H. M. Dao, „Offline KI,“ Osnabrück, 2025.

Anhang A

Der Anhang enthält ergänzende Materialien zur vorliegenden Arbeit, die aus Gründen der Übersichtlichkeit nicht im Hauptteil dargestellt wurden. Dazu gehören insbesondere der Quellcode der entwickelten Anwendung sowie ein Demonstrationsvideo zur Veranschaulichung der Funktionalität des Prototyps.

Auf dem beigefügten USB-Stick sind folgende Ordner und Dateien enthalten.

Tabelle A.1: Inhalt des Datenträgers

Pfad	Dateien	Beschreibung
\Bachelorarbeit_HaiMy_Dao	Bachelorarbeit_HaiMy_Dao.pdf	Die Bachelorarbeit im Portable Document Format (PDF)
	Bachelorarbeit_HaiMy_Dao.doc	Die Bachelorarbeit im Microsoft Word Format
\Bachelorarbeit_HaiMy_Dao	KontaktberichtErstellen.mp4	Demonstrationsvideo zur Funktionsweise des Prototyps.
\Bachelorarbeit_HaiMy_Dao \AudioKit	---	Enthält das Modul zur Audioverarbeitung.
\Bachelorarbeit_HaiMy_Dao \AudioKit\AudioKit	AudioKit.swift AudioKitFFI.swift AudioSTT.swift AudioTTS.swift KontaktberichtAudioService.swift	Enthält die Implementierung der Audioverarbeitungskomponenten.
\Bachelorarbeit_HaiMy_Dao \KontaktberichtLogikKit	---	Modul zur Analyse von Texten und Erstellung von Kontaktberichten.

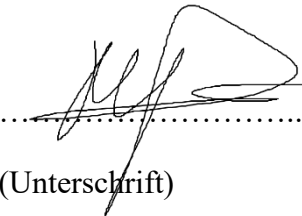
\Bachelorarbeit_HaiMy_Dao \KontaktberichtLogikKit \KontaktberichtLogikKit \Models	Ansprechpartner.swift Kontaktbericht.swift KontaktberichtStore.swift	Modellklassen zur Abbildung der Datenstruktur von Kontaktberichten und zugehörigen Entitäten.
\Bachelorarbeit_HaiMy_Dao \KontaktberichtLogikKit \KontaktberichtLogikKit \Services	KontaktberichtLLMService.swift KontaktberichtSpeicherService.swift	Komponenten zur Verarbeitung und Speicherung von Kontaktberichten sowie zur Anbindung des KI-Sprachmodells.
\Bachelorarbeit_HaiMy_Dao \KontaktberichtLogikKit \KontaktberichtLogikKit \Supports	ExtractDateFromText.swift StripDateAndTime.swift BerichtTool.swift LastBerichtTool.swift NeedClarificationTool.swift	Hilfskomponenten zur Unterstützung der Textanalyse, einschließlich Datenextraktion und regelbasierter Verarbeitung.
\Bachelorarbeit_HaiMy_Dao \kontaktbericht_swift_flutter	---	Flutter-Anwendung mit Benutzeroberfläche und FFI-Anbindung.
\Bachelorarbeit_HaiMy_Dao \kontaktbericht_swift_flutter \lib	gespraechstyp_item.dart kontaktbericht_form_page.dart kontaktbericht_list_page.dart audio_channel.dart kontaktbericht_ffi.dart permissions_service.dart section_header.dart	Enthält den Dart-Code der Anwendung, einschließlich Benutzeroberfläche, Logik und FFI-Anbindung.
\Bachelorarbeit_HaiMy_Dao \kontaktbericht_swift_flutter \native_audio	---	Enthält die native Audioverarbeitung sowie zugehörige Plattform- und FFI-Bindings.

\Bachelorarbeit_HaiMy_Dao \kontaktbericht_swift_flutter \native_audio\ios \AudioKit.xcframework	---	Kompilierte native Bibliothek (XCFramework) zur Integration der Audiofunktionalität in iOS.
\Bachelorarbeit_HaiMy_Dao \kontaktbericht_swift_flutter \native_audio\lib	native_audio.dart native_audio_bindings_generated.dart	Dart-seitige FFI-Bindings zur Anbindung der nativen Audiofunktionen.
\Bachelorarbeit_HaiMy_Dao \kontaktbericht_swift_flutter \native_audio\src	native_audio.h	Header-Dateien und native Schnittstellen zur Bereitstellung der Audiofunktionen.
\Bachelorarbeit_HaiMy_Dao \kontaktbericht_swift_flutter \native_kontaktbericht	---	Enthält die native Logik zur Analyse von Texten und Erstellung von Kontaktberichten.
\Bachelorarbeit_HaiMy_Dao \kontaktbericht_swift_flutter \native_kontaktbericht\ios \KontaktberichtLogikKit.xcframework	---	Kompilierte native Bibliothek (XCFramework) zur Integration der Kontaktbericht-Logik in iOS.
\Bachelorarbeit_HaiMy_Dao \kontaktbericht_swift_flutter \native_kontaktbericht\lib	native_kontaktbericht.dart native_kontaktbericht_bindings_generated.dart	Dart-seitige FFI-Bindings zur Nutzung der nativen Kontaktbericht-Funktionen.
\Bachelorarbeit_HaiMy_Dao \kontaktbericht_swift_flutter \native_kontaktbericht\src	native_kontaktbericht.h	Header-Dateien und Schnittstellen der nativen Kontaktbericht-Komponente.

Eidesstattliche Erklärung

Ich erkläre hiermit an Eides statt, dass ich die vorliegende Arbeit selbstständig und ohne Benutzung anderer als der angegebenen Hilfsmittel angefertigt habe; die aus fremden Quellen direkt oder indirekt übernommene Gedanken sind als solche kenntlich gemacht. Die Arbeit wurde bisher in gleicher oder ähnlicher Form keiner anderen Prüfungsbehörde vorgelegt und auch noch nicht veröffentlicht.

Osnabrück, den06.04.2026.....

..........
(Unterschrift)